

Trabajo Práctico 1

Arquitectura en Software

2do Cuatrimestre de 2026



Alumno	Padrón
Nicolas Khalil Chaia	110768
Gonzalo Nicolas Crudo	110816
Dante Finci	108459
Ezequiel Martín Aragón	110643

Profesores: Christian Calónico, Guillermo Rugilo y Mariano D'Ascanio

Introducción

arVault es una fintech que ofrece una billetera digital con soporte para múltiples monedas y operaciones de cambio entre ellas. Debido al rápido crecimiento del servicio y a que su desarrollo prioriza la incorporación de funcionalidades por sobre la robustez de la arquitectura, comenzaron a surgir problemas que afectaron la experiencia de los usuarios y la reputación de la compañía.

En este trabajo se analizará la arquitectura, el código y la infraestructura del servicio de cambio de monedas, identificando los principales atributos de calidad involucrados y estudiando cómo las decisiones de diseño existentes los afectan. A partir de pruebas de carga y métricas, se propondrán e implementarán modificaciones utilizando tácticas arquitectónicas, evaluando posteriormente su impacto y los posibles trade-offs entre los distintos atributos de calidad.

Arquitectura y código recibido

La aplicación encargada del servicio se sitúa en el directorio `app/` y tiene unos pocos módulos de estructura. No existe una separación explícita de capas, sino que la responsabilidad se reparte entre tres archivos principales colaborando en un mismo proceso de Node.js. Además, todo el estado del sistema vive en RAM del proceso, y se sincroniza a disco de forma periódica y asíncrona.

`app.js` - interfaz HTTP:

Es el punto de entrada de la aplicación. Inicializa Express, registra las rutas del API y realiza una validación mínima de los cuerpos de solicitud (existencia de campos). Delega la lógica de negocio en el módulo de cambio y se limita, en lo esencial, a traducir peticiones HTTP en llamadas a funciones y a devolver respuestas JSON con los códigos de estado correspondientes.

`exchange.js` — lógica de negocio:

Concentra las operaciones sobre cuentas internas, tasas de cambio y el registro de transacciones, así como la orquestación de `POST /exchange`. En este módulo se resuelve también la dependencia de transferencias, implementada como una función interna simulada. El estado con el que trabaja (cuentas, tasas y log) se obtiene del módulo de persistencia al inicializarse el servicio y se mantiene en memoria a lo largo de la ejecución.

state.js — persistencia:

Al iniciar el servidor, su función es cargar el contenido de los archivos JSON del directorio state/ hacia estructuras de memoria para que el resto de la aplicación opere sobre ellas. La escritura a disco no es inmediata tras cada operación: se programa de forma periódica mediante intervalos (accounts y log aproximadamente cada 1 s; rates aproximadamente cada 5 s, según la implementación recibida). Así, el sistema de archivos actúa como respaldo diferido del estado volátil del proceso.

En conjunto, la organización del código corresponde a un monolito in-process: interfaz HTTP, reglas de negocio y acceso al estado conviven en un único proceso, con persistencia basada en archivos y sin base de datos externa. En el despliegue recibido existe una sola instancia de este proceso; el estado no se comparte con otras réplicas.

POST /exchange

La operación central del servicio es POST /exchange, orquestada en exchange.js tras la validación mínima de campos en app.js. El flujo, en la versión recibida, es el siguiente.

1. Se obtienen del cuerpo de la solicitud la moneda origen, la moneda destino, el monto en moneda origen y los identificadores de las cuentas de cliente involucradas.
2. Se consulta en memoria la tasa de cambio vigente entre ambas monedas y se calcula el monto equivalente en la moneda destino.
3. Se localizan las cuentas internas de la empresa correspondientes a cada moneda (no las cuentas del cliente).
4. Se verifica que la cuenta interna de la moneda destino tenga saldo suficiente para entregar el monto calculado. No se valida el saldo de las cuentas del cliente: el diseño asume que esa comprobación ocurre fuera del servicio.
5. Si hay fondos internos, se invoca la transferencia simulada desde la cuenta del cliente en moneda origen hacia la cuenta interna correspondiente; luego, desde la cuenta interna

en moneda destino hacia la cuenta del cliente destino. Cada invocación introduce una demora de entre 200 y 400 ms.

6. Si la segunda transferencia falla, se intenta devolver el monto original al cliente (compensación). Si la primera falla, la operación se registra como no exitosa sin alterar los saldos internos.

7. Ante el éxito de ambas transferencias, se actualizan en memoria los saldos de las cuentas internas (ingreso en la moneda origen y egreso en la moneda destino).

8. En todos los casos se agrega una entrada al log en memoria (éxito o fracaso, con observación si corresponde) y se responde al cliente con el código HTTP que corresponda.

Este flujo deja en evidencia tres rasgos del caso base: dependencia de latencia en las transferencias simuladas, mutación de estado compartido en memoria sin mecanismos explícitos de exclusión mutua, y registro de la operación en un log cuya persistencia a disco es diferida.

Infraestructura recibida

El sistema recibido corre íntegramente en un único servidor orquestado con Docker Compose, con los siguientes servicios:

API

La aplicación Node.js + Express que expone el servicio de cambio de monedas. Se construye desde app/Dockerfile.

Nginx

Actúa como reverse proxy único, expuesto en el puerto 5555, y redirige todo el tráfico a una única URL.

Graphite

Reciben y almacenan métricas de series de tiempo, de la aplicación y de los contenedores (CPU, memoria).

Grafana

Visualización de las métricas que provee Graphite.

Componentes y Conectores

En la versión recibida, el servicio se organiza de forma que las solicitudes HTTP atraviesas un proxy inverso antes de llegar a la aplicación de negocio. El estado de dominio (cuentas internas, tasas de cambio y registro de operaciones) se sostienen en el contexto y proceso de la API, donde persiste periódicamente en el sistema de archivos. La transferencia de fondos es simulada dentro de la aplicación y no es desplegada como un servicio particular.

Componentes:

Cliente: Donde nacen las peticiones HTTP hacia el servicio. Representa a los consumidores de la API

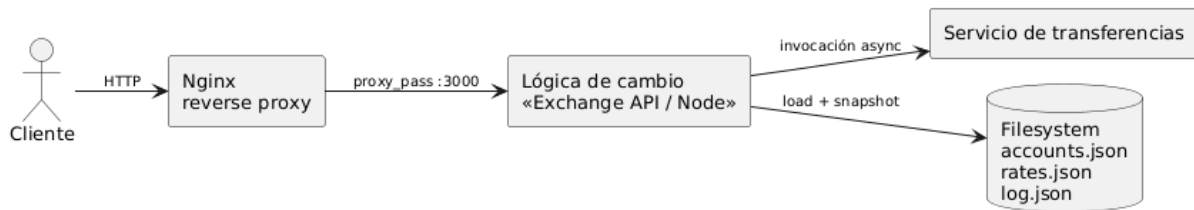
Nginx: Proxy inverso que expone el punto de entrada del servicio y reenvía el tráfico hacia la instancia de la API.

Exchange API: Aplicación que implementa la lógica del cambio de monedas. Consulta y actualiza las tasas y saldos de cuentas internas. Además, ejecuta operaciones de cambio y consulta en el log. Se ejecuta como único servicio dentro de un contenedor.

Almacén en sistema de archivos: Conjunto de archivos JSON (accounts.json, rates.json y log.json) utilizados como medio de persistencia del estado. No existe, en el caso base, una base de datos externa.

Servicio de transferencias: Dependencia lógica usada durante POST /exchange para modelar el movimiento de fondos. En la implementación actual se resuelve con una espera

aleatoria de entre 200 y 400 ms y un resultado siempre exitoso; no corresponde a un contenedor ni a un proceso independiente.



Las interacciones principales son sincrónicas sobre HTTP entre Cliente, Nginx y Exchange API. Entre la API y el almacén en archivos, la comunicación es por lectura al iniciar y escrituras periódicas del estado en memoria. Entre la lógica de cambio y el servicio de transferencias simulado, la interacción es una invocación interna asíncrona con demora artificial.

Atributos de calidad clave

Teniendo en cuenta el objetivo de arVault, el sistema debe estar orientado a cumplir con tres atributos claves y todas las reglas de negocios se deben realizar en pos de que estos se respeten.

Consistencia

Este atributo es clave debido a que unas de las motivaciones de la empresa para contratarnos fue que surgieron “reclamos de los usuarios sobre problemas en el uso de la función de cambio de monedas”. En un sistema financiero nunca se debe dejar al azar que este atributo se cumpla o no ya que puede llevar a la quiebra de la compañía y juicios.

Performance.

Otro de los atributos claves para un sistema financiero debe ser la performance ya que este le permite competir contra otros sistemas. Siendo eficiente los usuarios les agrada la experiencia de uso y volverán a utilizar la plataforma.

Disponibilidad y fiabilidad.

Dado que su fundador busca que la empresa crezca de manera significativa, arVault no puede permitirse que cualquier falla del proceso deje al servicio completo caído. Sin contar que al caerse el sistema puede ser que queden operaciones a mitad de camino creando una ventana de pérdida de datos, lo que puede generar que se debiten dos veces un mismo monto o una transacción nunca se concluya.

Todos los atributos de calidad

Escalabilidad

Si el plan de la fintech es que la cantidad de usuarios crezcan para sostener la inversión recibida. Es clave que el diseño del sistema permita un escalado con el paso del tiempo.

Seguridad

Siendo que se opera con datos privados de los usuarios es muy importante que la empresa maneje alta seguridad en sus comunicaciones y en la forma de almacenar sus datos, es por esto que se debe validar todo lo que entre y salga del sistema, como también analizar posibles errores para tener siempre una forma de atraparlos y que eso no genera una mala experiencia en el usuario, ni tampoco permitir que se sobrecargue la red generando que se caiga el sistema perdiendo disponibilidad.

Testeabilidad

Todo el sistema recibido no contaba con ningún tipo de tests, ni unitarios ni end to end. Además, dada la arquitectura de forma monolítica es más complicado aún agregarlos debido a que hay mucho acoplamiento.

Modificabilidad

Como antes fueron analizados y descritos los módulos, de la manera actual no hay una separación de responsabilidades clara, por ejemplo, el manejo de saldos de cuentas está en la misma lógica que calcula tasas y ejecuta transferencias. Al no haber capas que

diferencien lo que hace cada elemento, no hay una lógica de negocio ni acceso a datos estructurados.

Portabilidad

Este atributo ya fue bien resuelto y entregado con éxito por los antiguos desarrolladores. Se utiliza Docker y Docker Compose creando contenedores que encapsulan las dependencias.

Interoperabilidad

El lado bueno en este atributo es que se utiliza una API REST sobre HTTP/JSON, esto es lo habitual hoy en día por lo tanto cualquier cliente podrá comunicarse y utilizar la APP. Sin embargo no se cuenta ni con un versionado de la API ni documentación formal sobre el código y los endpoints, esto dificulta la evolución del código a futuro.

Simplicidad

En el producto recibido este atributo es una realidad ya que solo cuenta con tres archivos y un flujo de negocio lineal, el código acotado y fácil de leer de punta a punta. El sacrificio de tener esta simplicidad es que dificulta los atributos de modificabilidad y testabilidad.

Usabilidad

Dado que este atributo se contempla desde el punto de vista de los clientes, hoy en día el sistema no es claro respecto a los errores. Esto se debe a que por un lado son genéricos y no explicitan que es lo que está fallando o no son los códigos de errores adecuados según los estándares.

Comportamiento del servicio

Con el objetivo de obtener un panorama objetivo del comportamiento del servicio en su estado original, se ejecutaron pruebas de carga y de estrés sobre cada uno de los seis endpoints expuestos por la API de arVault.

El entorno se fijó igual para todo el equipo: el contenedor de la API corre con ``cpus: "2.0" y `mem_limit: "1g" en Docker Compose, de modo que los resultados sean comparables entre integrantes y entre corridas.`

Pruebas de carga

Se definieron escenarios individuales por endpoint, simulando patrones de consumo representativos de situaciones reales. Para las pruebas de estrés se usó una configuración de escalones progresivos de 60 segundos cada uno (50, 100, 200, 400, 800 y 1600 req/s para los endpoints de cuentas, tasas y log; 10, 25, 50, 100, 200 y 400 req/s para `/exchange`. Todos cuentan con una fase de calentamiento inicial y una fase final de recuperación a carga baja, permitiendo ubicar en qué escalón aparece la degradación.

Servicio GET /accounts

Escenario de carga: lectura frecuente del listado de cuentas, con un pool de 50 conexiones compartidas, calentamiento de 20 s a 2 req/s, rampa de 60 s de 2 a 25 req/s, pico de 90 s a 25 req/s y recuperación de 30 s a 5 req/s (220 s en total). De las 3250 requests ninguna fallo y los tiempos de respuesta fueron despreciables

Escenario de estrés: pool de 300 conexiones, calentamiento de 30 s a 20 req/s y escalones de 60 s a 50, 100, 200, 400, 800 y 1600 req/s, con recuperación final de 60 s a 25 req/s (450s en total).

El servicio absorbe todo lo ofrecido sin un solo timeout hasta 800 req/s, donde recién asoma cola en el p99 (9 ms) y en el máximo (97 ms). El quiebre aparece entre 800 y 1600 req/s con 1808 timeouts y un p99 de 3,4 s. La recuperación es inmediata: en cuanto baja la carga, la latencia vuelve a 1 ms.





Servicio GET /rates

Escenario de carga: mismas fases que GET /accounts. De los 3250 requests ningún fallo.

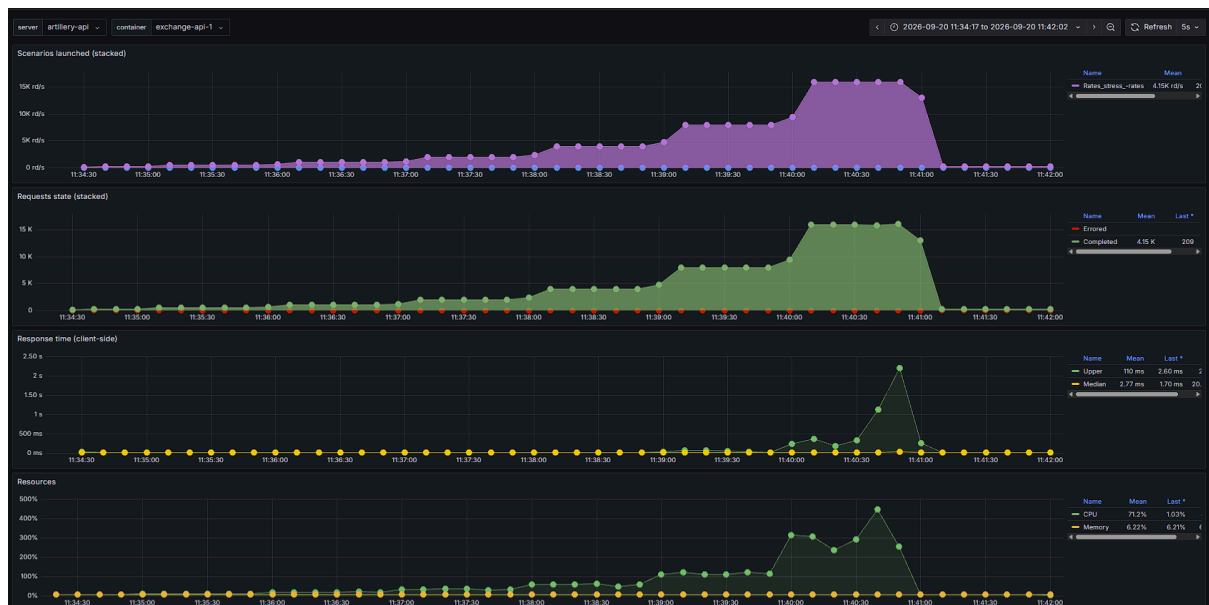
Escenario de estrés: mismos escalones que el anterior de 50 a 1600 req/s.

A diferencia del anterior, este endpoint no tuvo un solo timeout en ningún escalón, incluido

1600 req/s. Sí se degrada la latencia (p99 de 330 ms, máximo de 2,2 s), pero sin colapsar.

La hipótesis es que la response de este endpoint pesa mucho menos que la del anterior.





Servicio GET /log

Escenario de carga: Este endpoint se ejecutó sobre un log poblado con 390 transacciones previas (generadas por una corrida de POST /exchange), lo que da una respuesta de 95 KB.

De las 3250 requests sin ninguna fallo, con media de 2 ms, mediana de 2 ms, p95 de 3 ms y p99 de 4 ms.

Escenario de estrés: con log poblado y mismos escalones.

Con 390 entradas, el techo de este endpoint cae de forma drástica: colapsa ya en el escalón de 400 req/s (382 timeouts, p99 de 4,1 s). A 800 req/s el 57,7 % de las 191.100 requests de toda la corrida terminaron en fallo (timeouts, resets de conexión o agotamiento de puertos del generador).

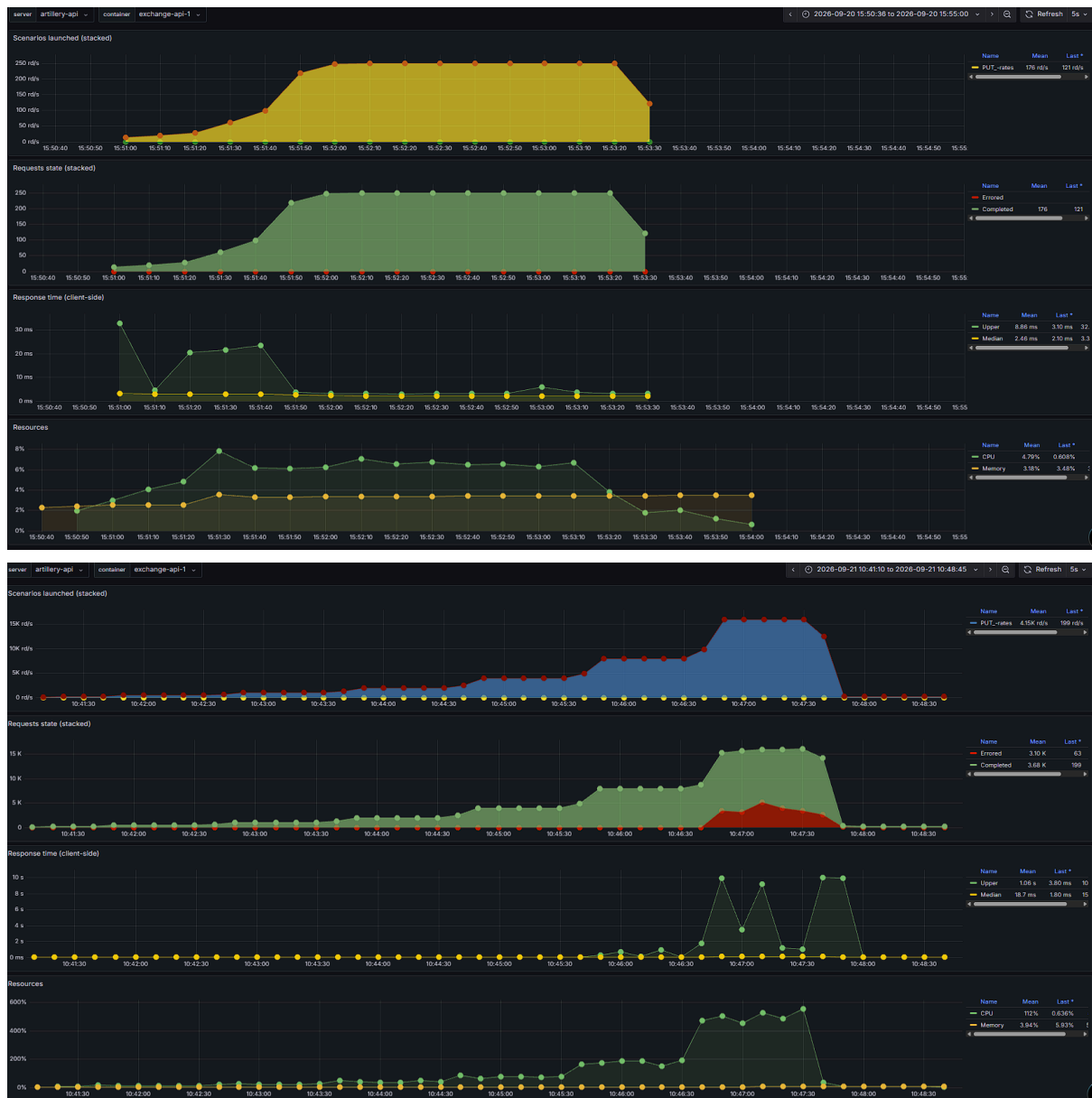


Servicio PUT /rates

Escenario de carga: mismas fases que los anteriores, enviando en cada request el valor de tasa ya vigente (USD/ARS = 1513) para no alterar el estado. Se procesaron 3250 requests sin ningún fallo, con media de 1,2 ms, mediana de 1 ms y p99 de 2 ms.

Escenario de estrés: mismos escalones que GET /accounts (pool de 300, 50 a 1600 req/s), recreando el contenedor api antes de la corrida.

En el escalón de 800 req/s la escritura ya se degrada ahí (p99 de 176 ms, máximo de 1,8 s), aunque sin fallos. En 1600 req/s colapsa con 9014 timeouts más 12.614 resets de conexión (21.628 fallos totales sobre 96.000 requests ofrecidas en el escalón).



Servicio PUT /accounts/:id/balance

Escenario de carga: mismas fases que los gets anteriores, enviando en cada request el balance ya vigente de la cuenta 1, para no alterar el estado. Se procesaron 3250 requests sin ningún fallo, con media de 1,2 ms, mediana de 1 ms y p99 de 2 ms.

Escenario de estrés: mismos escalones que `GET /accounts`, recreando el contenedor antes de la corrida.

Sucede algo similar al endpoint de PUT /rates donde la degradación ya visible a 800 req/s (p99 de 213 ms, máximo de 4,5 s, todavía sin fallos) y colapso a 1600 req/s con 21.812 fallos totales, incluyendo 11 respuestas HTTP 502 con latencias de 7 a 10 s. Completó un 88,6 % de las 191.100 requests de la corrida.



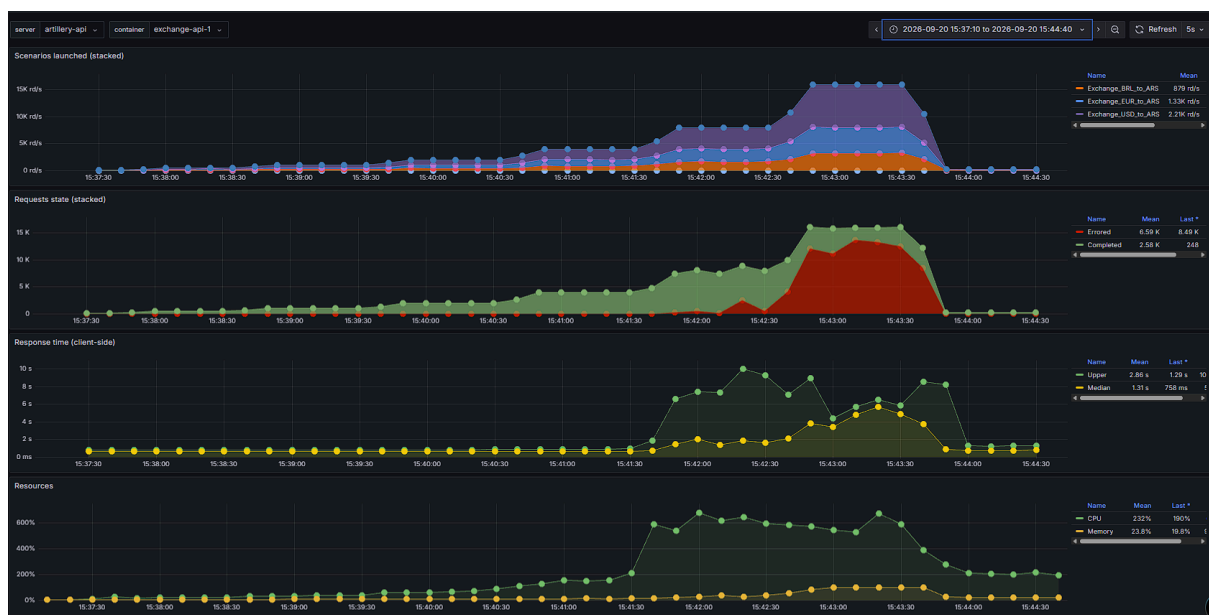
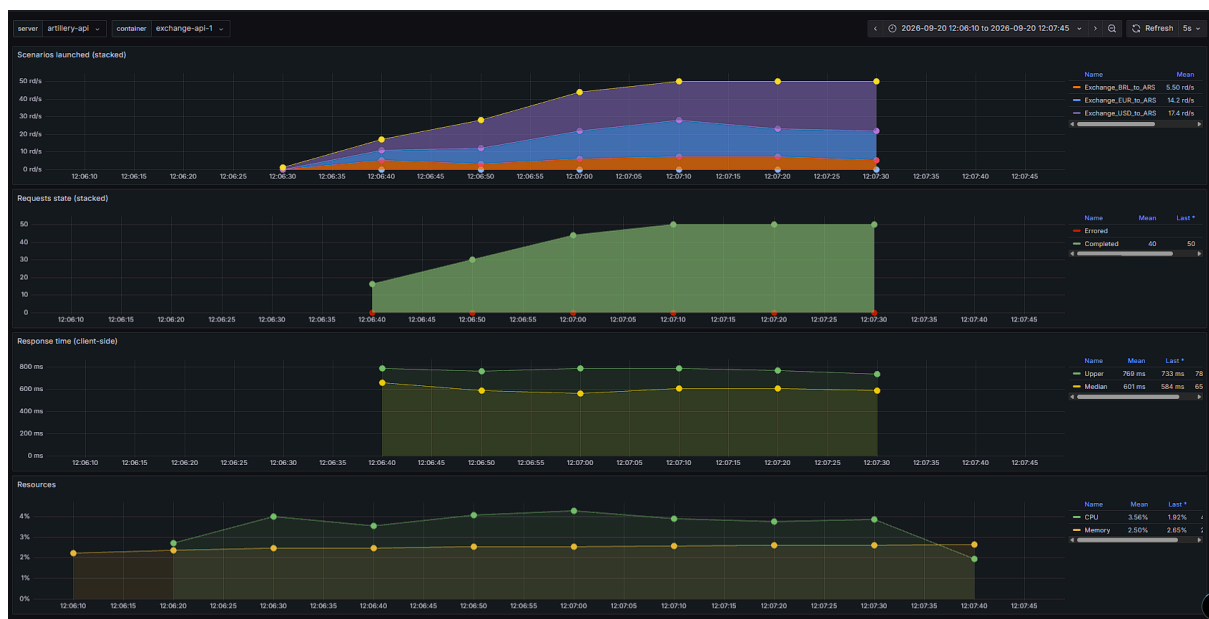
Servicio POST /exchange

Escenario de carga: ciclo de operaciones de cambio mezclando USD, EUR y BRL contra ARS, con un pool de 50 conexiones, rampa de 30 s de 1 a 5 req/s y carga sostenida de 60 s a 5 req/s. Antes de la corrida se fija el saldo interno de la cuenta en ARS en un valor alto, para que ninguna operación falle por falta de fondos durante la prueba.

Se procesaron 390 requests sin ningún fallo, con media de 599 ms, mediana de 596 ms, mínimo de 426 ms y máximo de 783 ms. Este piso de latencia se debe a las dos transferencias simuladas, de entre 200 y 400 ms cada una, que se ejecutan en secuencia.

Escenario de estrés: dada la latencia y el costo del endpoint primero se usó hizo un pool de 1000 conexiones, calentamiento de 30 s a 5 req/s y escalones de 60 s a 10, 25, 50, 100, 200 y 400 req/s, con recuperación final a 25 req/s. Sin embargo, el servicio no llegó a

romperse y la latencia se mantuvo estable. Por lo tanto se decidió realizar una segunda corrida subiendo la escala a 50, 100, 200, 400, 800 y 1600 req/s para encontrar el quiebre real. En esta segunda corrida se descubrió que el quiebre se ubica entre 400 y 800 req/s: a 400 req/s ya hay señales de cola (p95 de 873 ms, máximo de 1,8 s) sin fallos todavía; a 800 req/s la mediana se triplica (1,7 s), el p99 llega a 7,4 s y aparecen 529 timeouts más 7223 resets, a 1600 req/s el servicio queda prácticamente colapsado, completando solo una fracción de lo ofrecido. A diferencia de los demás endpoints, la degradación de `/exchange` no está asociada a saturación de CPU. La recuperación no fue completamente limpia: al volver a 25 req/s, la mediana quedó en 756 ms, contra 600 ms de línea base.



Tácticas aplicadas

Táctica 1: Modularización

Motivación

Principalmente consistió en organizar el código para que toda la construcción a futuro de la aplicación sea más sencilla y organizada. Acá no se buscó que hubiera impacto en las métricas, sino que fue una inversión a futuro en mantenibilidad.

Táctica aplicada

Se modularizó el código creando nuevas carpetas como `api/`, `services/`, `repository/` y `exceptions/`, coordinado todo desde `app.js` como composition root de inyección de dependencias.

Predicción

Los resultados serían similares a los de main, o hasta podrían empeorar levemente, debido a que se agregan más saltos (`router` → `service` → `repository`) para obtener una respuesta.

Resultados

Se repitió exactamente la misma batería de pruebas que en el código original recibido.

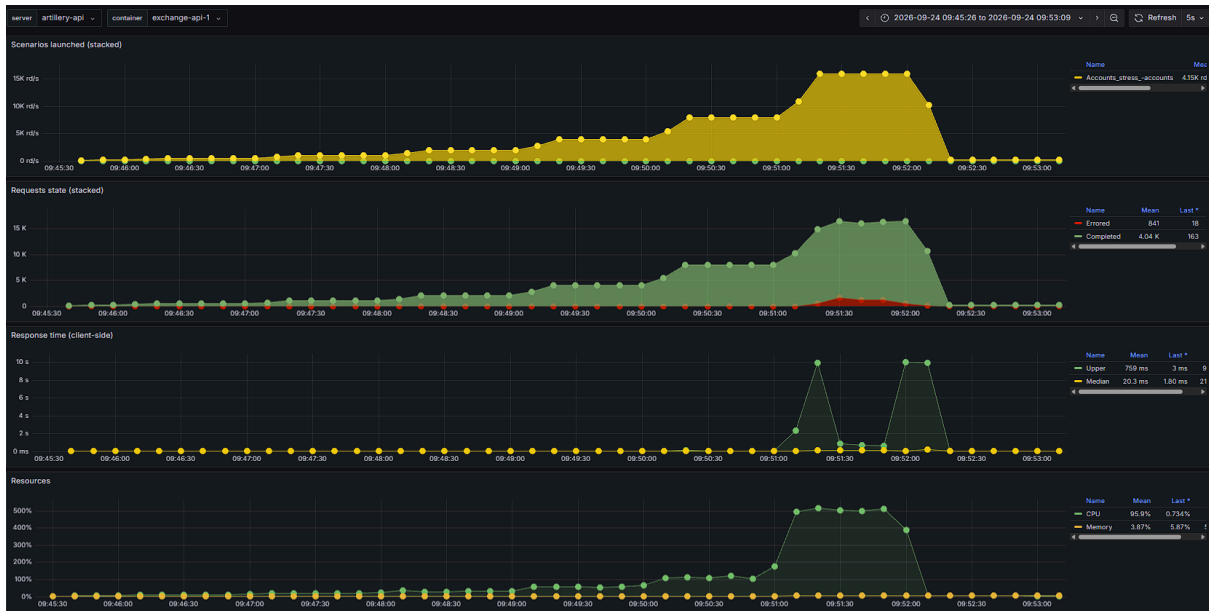
Servicio GET /accounts

En carga fue idéntico al original: 3250 requests, 0 fallos, media 1,2 ms, mediana 1 ms, p99 2 ms, máximo 11 ms (original: media 1,1 ms, p99 2 ms, máximo 3 ms). En estrés, hasta 800 req/s el comportamiento fue idéntico al original. En 1600 req/s, modular tuvo el triple de fallos: 5047 fallos totales contra los 1877 del original. La CPU del api fue similar en ambos casos (1,01 contra 0,95 cores), así que no es un problema de cómputo por request, lo que cambia es cuántas requests terminan colgadas o cortadas antes de ser atendidas. Es razonable atribuirlo a las capas de indirección que agregó la modularización, invisibles a carga normal pero medibles en el borde exacto del colapso, donde cada milisegundo extra empuja a más requests a cruzar el timeout de 10 s.

GET /accounts (carga)



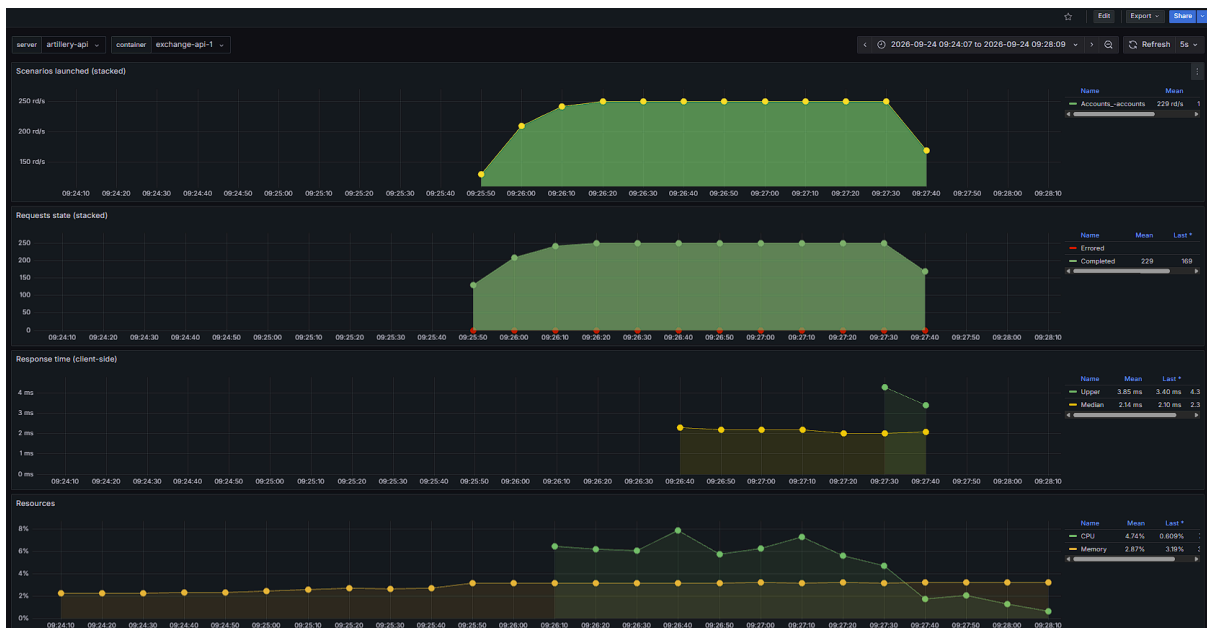
GET /accounts (estrés)



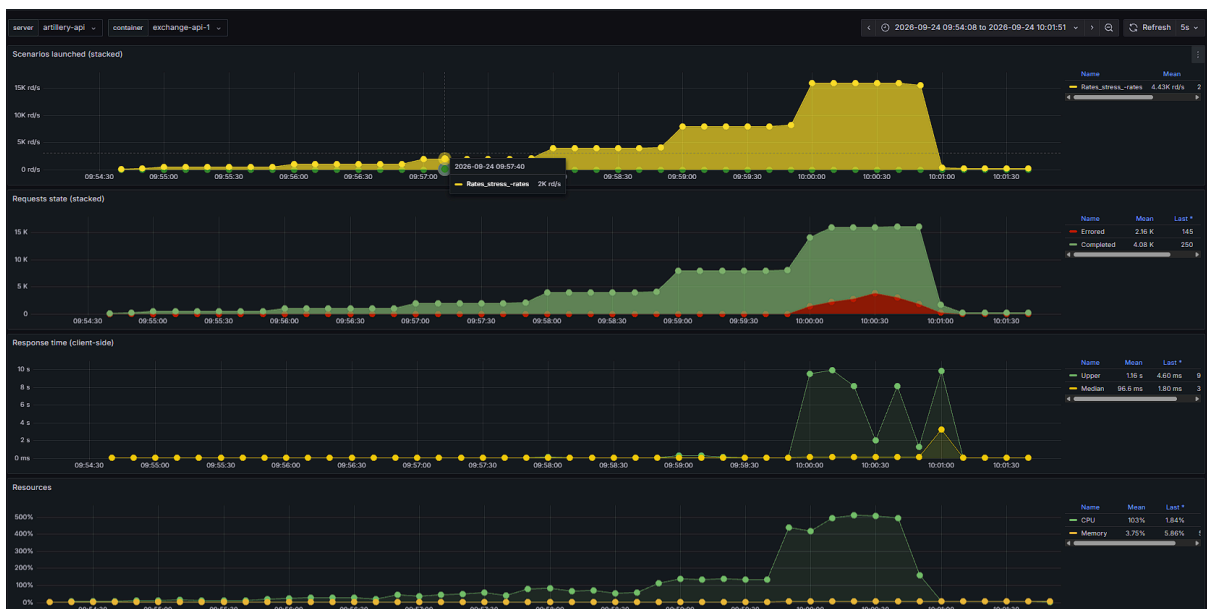
Servicio GET /rates

En carga fue idéntico al original: 3250 requests, 0 fallos, media 1,1 ms, mediana 1 ms, p99 2 ms, máximo 30 ms (original: media 1 ms, p99 2 ms, máximo 4 ms). Esta fue la diferencia más marcada, el mismo escalón de 1600 req/s que el original toleró sin un solo fallo (p99 de 330 ms), en modular produjo 10.197 fallos (5239 timeouts + 4958 resets) más 2 respuestas HTTP 502, con p99 de 1,8 s. La recuperación tampoco fue limpia: con solo 25 req/s ofrecidos hubo 1886 fallos adicionales y una mediana de 675 ms, cuando en el original la latencia volvía a 1 ms apenas bajaba la carga.

GET /rates (carga)



GET /rates (estrés)

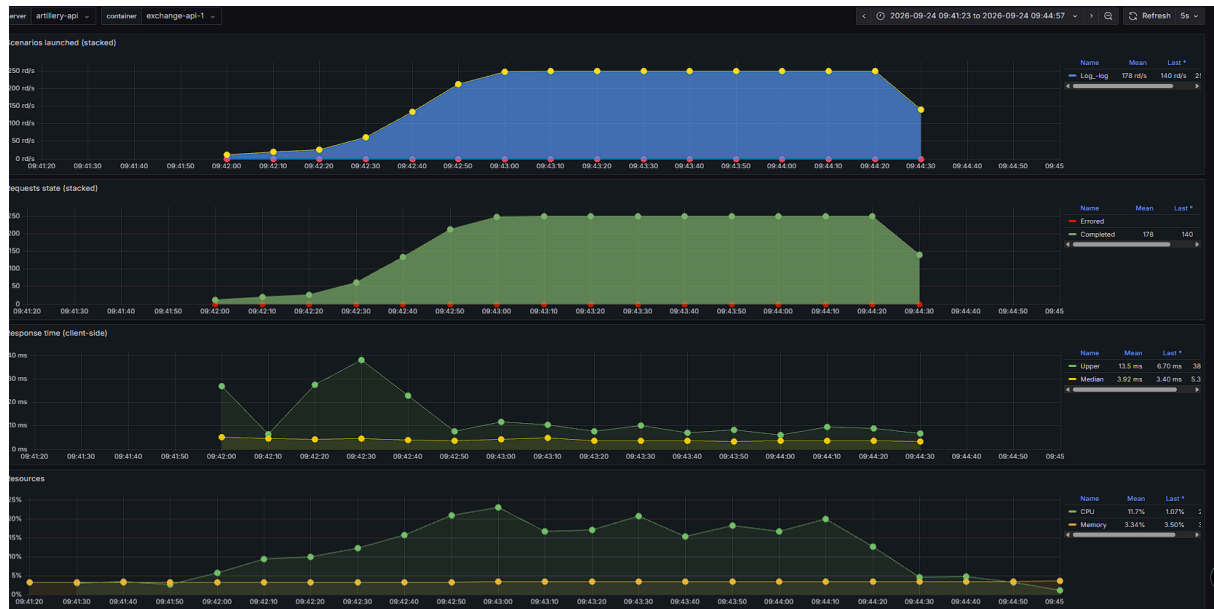


Servicio GET /log

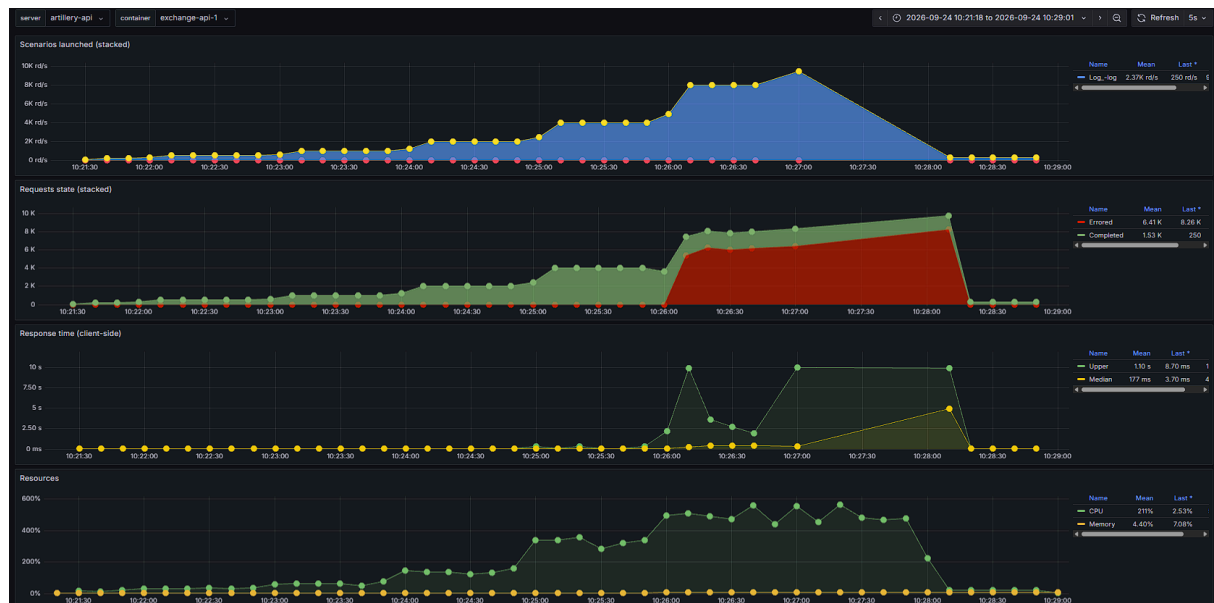
En carga, con el log poblado igual que en el original (~390 entradas): 3250 requests, 0 fallos, media 2,2 ms, mediana 2 ms, p95 3 ms, p99 4 ms, máximo 8 ms, igual que el original (media 2 ms, p99 4 ms, máximo 5 ms). En estrés, con ese mismo log poblado, el original ya colapsaba en 400 req/s (382 timeouts, p99 de 4,1 s); modular pasó ese escalón sin un solo fallo y recién se rompió en 800 req/s — el único caso de toda la batería donde modular aguanta claramente un escalón más antes de romperse. Aun así, el total de fallos de toda la corrida terminó siendo del mismo orden de magnitud: 119.790 fallos sobre 191.100 requests (62,7%) en modular contra 110.343 (57,7%) en el original. La causa de fondo no cambia con

la modularización: el costo de GET /log escala con el tamaño acumulado del historial, porque no hay paginación ni límite de crecimiento (esto se resuelve recién en la rama log-pagination, más adelante).

GET /log (carga, log poblado)



GET /log (estrés, log poblado)



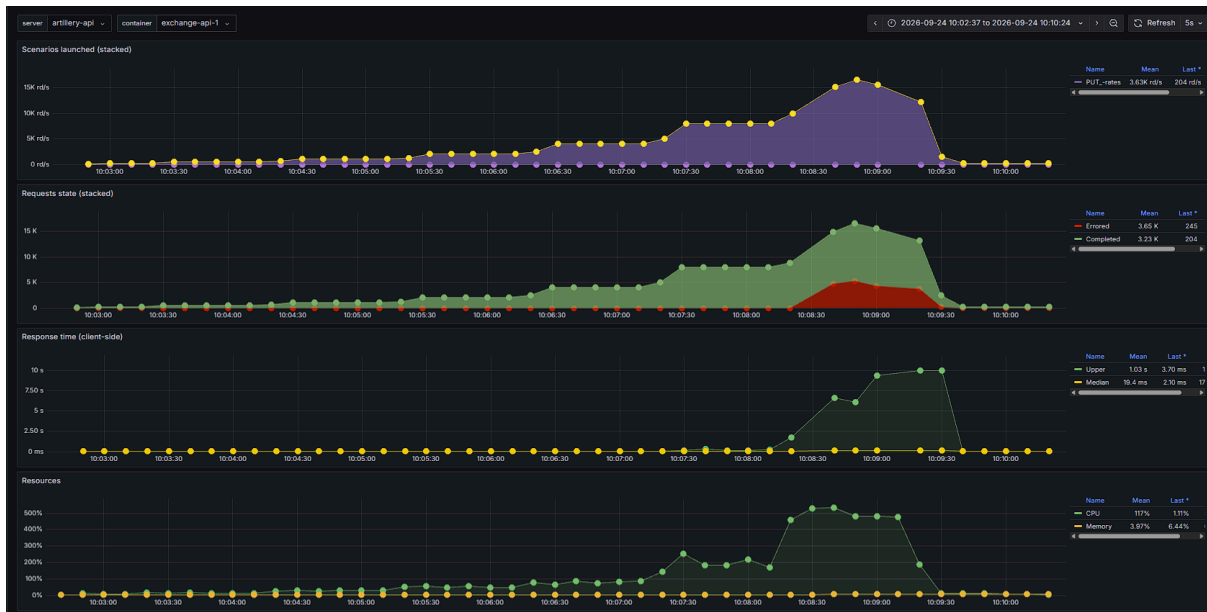
Servicio PUT /rates

En carga fue idéntico al original: 3250 requests, 0 fallos, media 1,1 ms, mediana 1 ms, p99 2 ms, máximo 10 ms (original: media 1,2 ms, p99 2 ms, máximo 21 ms). En estrés, en el escalón de 800 req/s modular se comportó mejor que el original: p99 de 26 ms contra 176 ms, máximo de 211 ms contra 1755 ms. En 1600 req/s los totales fueron similares en orden de magnitud (14.285 fallos parciales contra 21.628 en el original). Donde reaparece el patrón sistemático es en la recuperación: 3965 fallos con solo 25 req/s ofrecidos (contra 63 en el original), con la latencia disparada a p99 de 3,7 s.

PUT /rates (carga)



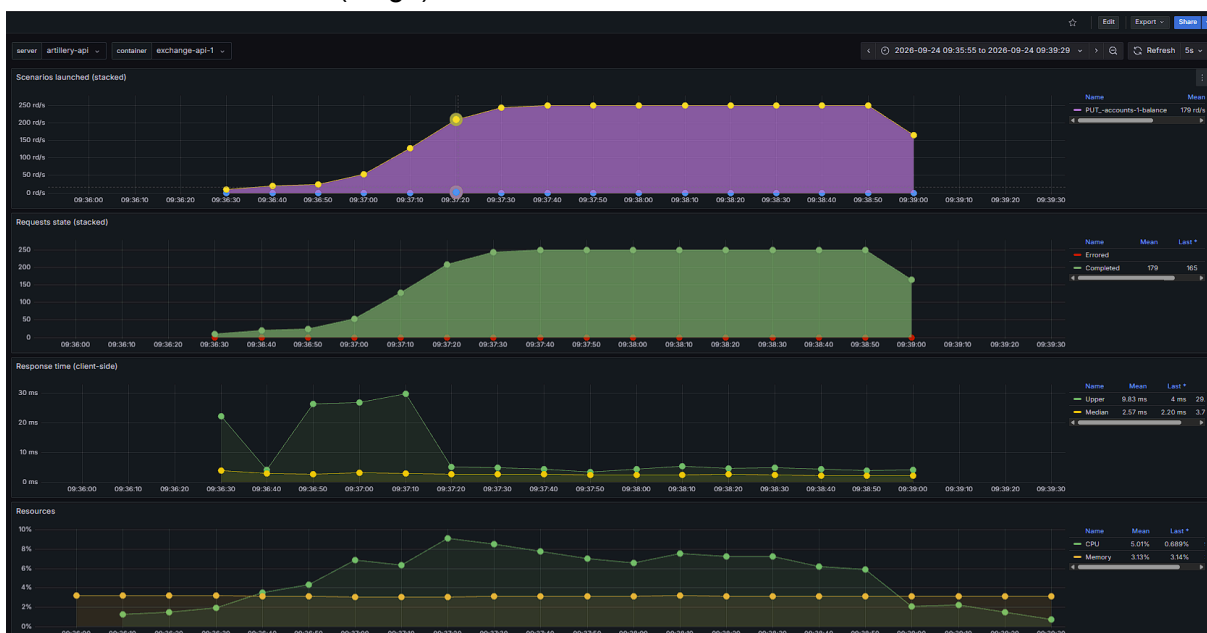
PUT /rates (estrés)



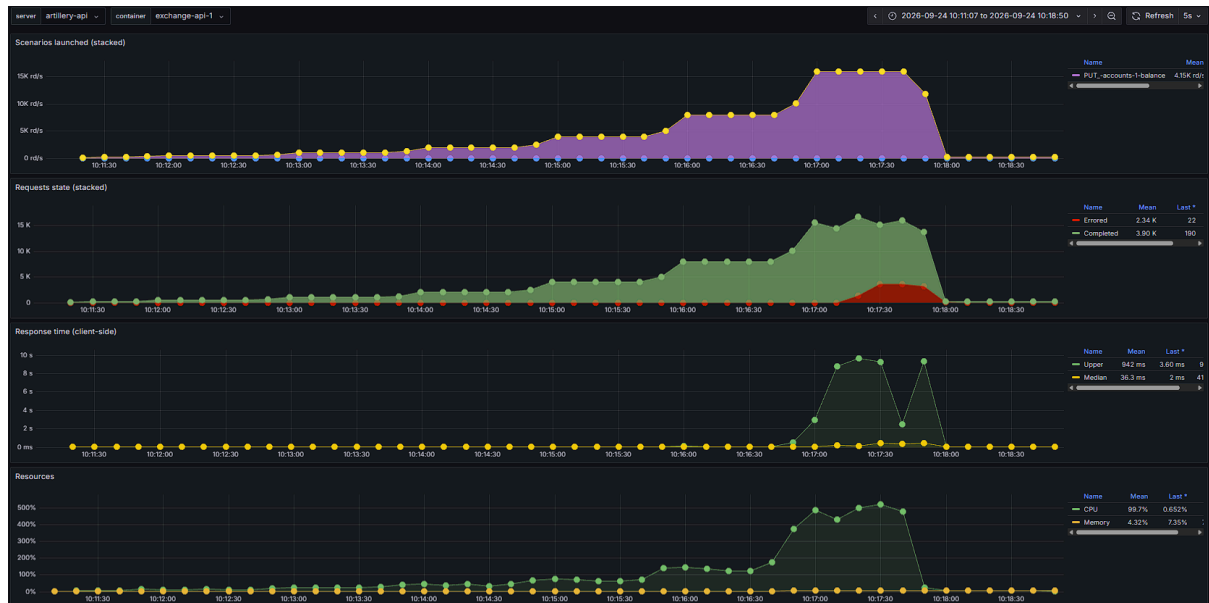
Servicio PUT /accounts/:id/balance

En carga fue idéntico al original: 3250 requests, 0 fallos, media 1,3 ms, mediana 1 ms, p99 2 ms, máximo 3 ms (original: media 1,2 ms, p99 2 ms, máximo 14 ms). Este fue el endpoint más favorable a modular de toda la batería: a 800 req/s, p99 de 10 ms contra los 213 ms del original, máximo de 97 ms contra 4462 ms. A 1600 req/s, 4877 fallos totales contra los 21.812 del original, menos de la cuarta parte, aunque con la CPU de nginx más alta (1,28 contra 0,65 cores), lo que sugiere que el cuello de botella se corrió hacia nginx. La recuperación, otra vez, no fue limpia (3255 fallos a 25 req/s).

PUT /accounts/1/balance (carga)



PUT /accounts/1/balance (estrés)



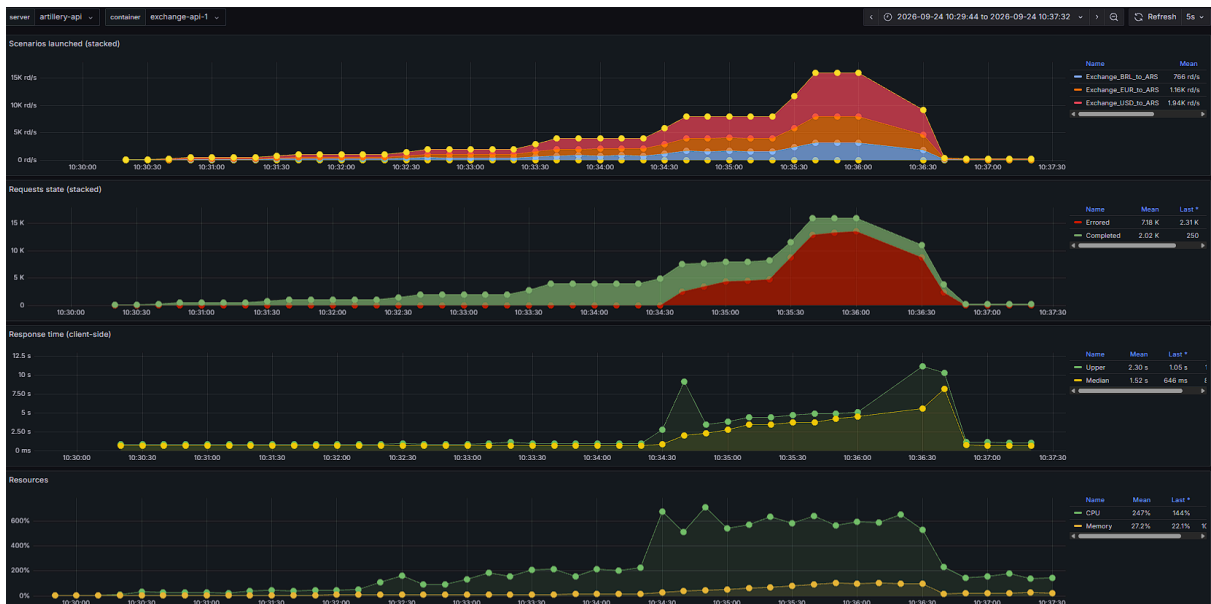
Servicio POST /exchange

En carga fue idéntico al original: 390 requests, 0 fallos, media 610 ms, mediana 608 ms, p95 743 ms, p99 773 ms, mínimo 422 ms, máximo 794 ms (original: media 599 ms, mediana 596 ms, rango 426-783 ms), consistente con el piso estructural de ~600 ms de las dos transferencias simuladas en secuencia (la modularización no tocó esa lógica). En estrés (escalones de 50 a 1600 req/s), el quiebre se ubicó en el mismo lugar que en el original (entre 400 y 800 req/s), y hasta 400 req/s el comportamiento fue similar. A partir de 800 req/s, modular terminó peor en el total de la corrida: 102.781 fallos contra los 79.112 del original, un 46,1% de éxito contra el 58,5% del original.

POST /exchange (carga, pobló el log)



POST /exchange (estrés)



Qué se puede afirmar

Bajo carga normal, no hay ninguna diferencia medible. Los seis endpoints dieron resultados estadísticamente idénticos al original, ya que la modularización reordenó el código (routers, services, repositories, manejo de excepciones) sin tocar la lógica de negocio de ningún endpoint.

Bajo estrés extremo, el resultado no es uniforme entre endpoints: 3 de 6 empeoraron (GET /accounts, GET /rates, POST /exchange), 1 mejoró claramente (PUT /accounts/:id/balance),

1 fue similar o algo mejor (GET /log) y 1 fue mixto (PUT /rates, mejor a 800 rps pero peor en recuperación). No hay un patrón de "modular siempre peor" ni "modular siempre mejor".

Lo más significativo es la recuperación: en las 6 pruebas de estrés de modular, la recuperación a carga baja después del pico de 1600 req/s nunca fue limpia, siempre quedaron cientos o miles de fallos y latencias de segundos arrastradas del escalón anterior. En el original, la recuperación fue inmediata en la mayoría de los casos (GET /accounts y GET /rates volvían a 1 ms apenas bajaba la carga).

Qué no se puede afirmar / matices

No se puede afirmar con certeza que las diferencias observadas (ni el empeoramiento en algunos endpoints, ni la mejora en otros, ni la recuperación degradada) estén causadas específicamente por las nuevas capas de indirección o por el manejo centralizado de errores, aunque es una hipótesis.

Trade-off

Gana	Pierde / empeora
Facilidad para modificar el proyecto a futuro	Simplicidad para explicar el código
Facilidad para extender el proyecto a futuro	Performance del sistema en estrés

Conclusión del caso

La modularización cumplió su objetivo declarado (mantenibilidad) sin costo medible bajo carga normal, que es el escenario relevante para la mayor parte de la vida operativa del servicio.

Táctica 2: Transacciones

Motivación

ExchangeService.exchange() (app/services/exchange.js) implementa un patrón check-then-act sin exclusión mutua, y la ventana entre el chequeo y la escritura es incluso más ancha de lo que un lost update típico sugeriría, porque en el medio hay operaciones asíncronas de cientos de milisegundos:

```
// app/services/exchange.js luego de modularizar el código antes de aplicar transacciones

// check si hay fondos en la cuenta de contrapartida
if (counterAccount.balance >= counterAmount) { // (1) LECTURA
  if (await transfer(clientBaseAccountId, baseAccount.id, baseAmount)) { // (2) ~200-400ms
    if (await transfer(counterAccount.id, clientCounterAccountId, counterAmount)) { // (3) ~200-400ms
      // recién acá se escribe el saldo
      this.accountsRepository.adjustAccountBalance(baseAccount.id, baseAmount); // (4) ESCRITURA
      this.accountsRepository.adjustAccountBalance(counterAccount.id, -counterAmount);
      ...
    }
  }
}
```

Entre (1) y (4) no hay ningún lock — de hecho, AccountsRepository (app/repository/accounts.js) no define ningún mecanismo de sincronización: solo getAccountById, getAccountByCurrency, setAccountBalance y adjustAccountBalance, todos operando directo sobre el array this.accounts en memoria. Y (2)/(3) son dos await transfer(...) que tardan entre 200 y 400 ms cada uno, así que la ventana de carrera no es del orden de microsegundos como en un lost update de libro: son cientos de milisegundos reales, tiempo de sobra para que múltiples requests concurrentes lean el mismo counterAccount.balance en (1), todas pasen el chequeo, y todas lleguen a (4) a descontar sobre una base que ya no refleja los descuentos de las otras. El resultado es una cuenta sobregirada o con saldo inconsistente sin que nada lo detecte. **El QA en juego es consistencia de datos / corrección bajo concurrencia, no rendimiento.**

Táctica aplicada

Se aplicó una táctica de gestión de concurrencia por exclusión mutua (locking), sobre el recurso compartido correcto (el par de cuentas), no sobre todo el servicio. El código en transactions agrega, en app/repository/[accounts.js](#):

```
async withAccountsLock(accountIds, fn) {
  const ids = [...new Set(accountIds)].sort(); // orden determinístico -> evita deadlock
  const releases = [];
  try {
    for (const id of ids) {
      releases.push(await this.acquireLock(id));
    }
    return await fn();
  } finally {
    for (const release of releases.reverse()) {
      release();
    }
  }
}

acquireLock(accountId) {
  const tail = this.locks.get(accountId) ?? Promise.resolve();
  let release;
  const held = new Promise((resolve) => { release = resolve; });
  this.locks.set(accountId, tail.then(() => held));
  return tail.then(() => release);
}
```

Es un mutex por cuenta implementado con promesas encadenadas (this.locks es un Map de accountId -> promesa de liberación), que ordena los ids antes de tomarlos ([...new Set(accountIds)].sort()). Ese orden fijo es lo que evita deadlock cuando dos exchanges concurrentes necesitan las mismas dos cuentas pero en direcciones opuestas (USD→ARS toma [ARS, USD] y ARS→USD también toma [ARS, USD], nunca [USD, ARS]).

Y en app/services/exchange.js, el chequeo de saldo y el ajuste de ambos saldos quedan dentro del lock, en vez de después de los await transfer(...):

```
const reserved = await this.accountsRepository.withAccountsLock(
  [baseAccount.id, counterAccount.id],
  () => {
    if (counterAccount.balance < counterAmount) {
      exchangeResult.obs = "Not enough funds on counter currency account";
      return false;
    }
    this.accountsRepository.adjustAccountBalance(baseAccount.id, baseAmount);
    this.accountsRepository.adjustAccountBalance(counterAccount.id, -counterAmount);
    return true;
  }
);

if (reserved) {
  if (await transfer(clientBaseAccountId, baseAccount.id, baseAmount)) {
    ...
  }
}
```

Es decir: el diseño no solo agrega el lock, sino que mueve el chequeo+ajuste de saldo a antes de las transferencias simuladas (reservando el saldo primero), sacando la parte lenta (transfer(), 200-400 ms) de la sección crítica. Esto es clave para el resultado de throughput: el lock se mantiene tomado solo durante el chequeo + dos escrituras en memoria (microsegundos), no durante los await transfer(). releaseReservation() (revertir la reserva si la transferencia externa simulada falla) toma el mismo lock, por la misma razón: también ajusta saldos que deben quedar serializados respecto de cualquier otro exchange sobre esas cuentas.

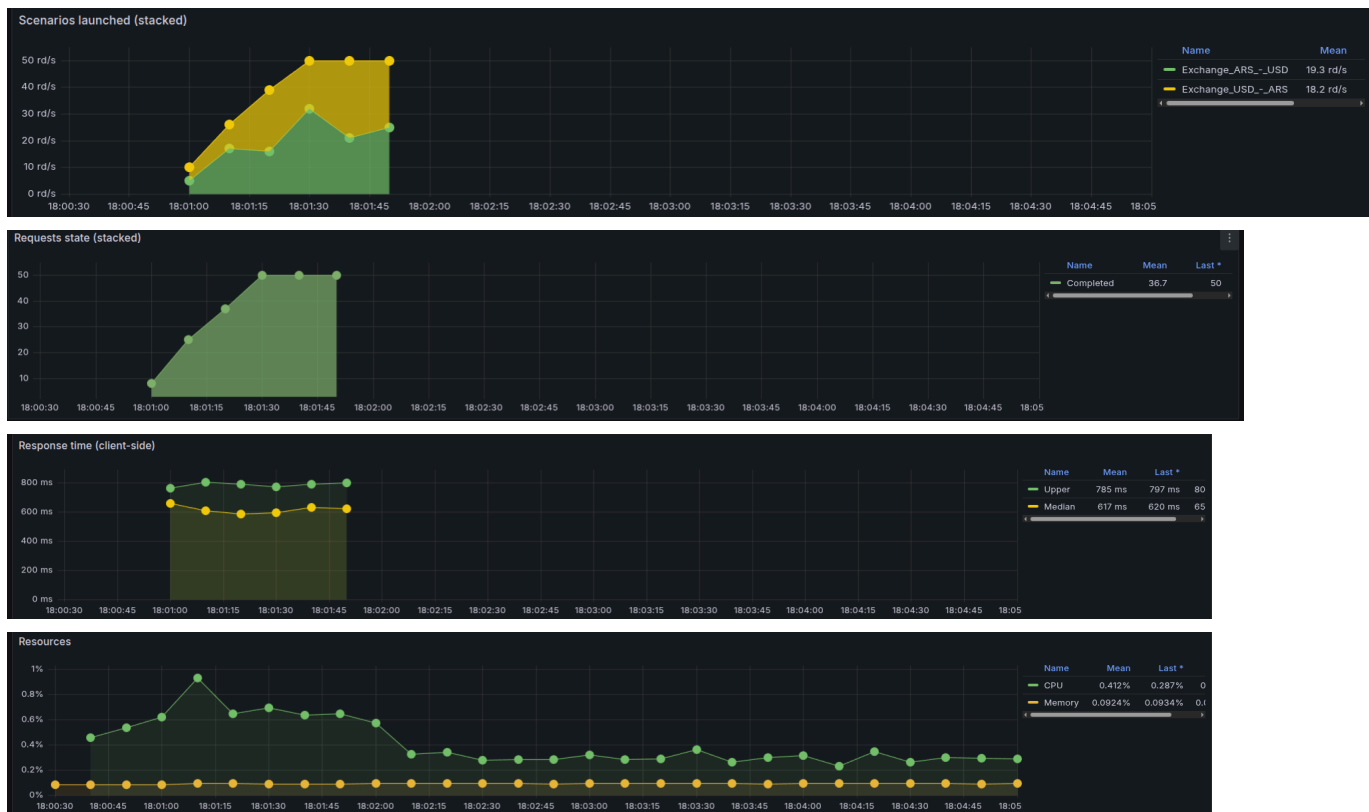
El cambio es acotado, no se tocó /accounts, /rates ni /log, porque no comparten el código de exchange() ni el estado que se busca proteger.

Predicción

- **Corrección:** bajo carga concurrente sobre el mismo par de cuentas, no debería volver a observarse ningún saldo negativo ni inconsistente, incluso a volúmenes de request muy por encima de los que veíamos en la versión anterior del código.
- **Latencia bajo carga baja:** sin contención real (pocos requests por segundo contra el mismo par de cuentas), el lock no debería agregar costo medible — el piso de latencia debería seguir siendo el mismo (~600 ms, dado por las dos transferencias simuladas de 200-400 ms en secuencia).
- **Throughput bajo estrés:** como el lock serializa todo exchange que toque un mismo par de cuentas, se espera un techo de capacidad bajo para ese par específico — muy por debajo de lo que puede pedir un escenario de estrés agresivo — y que superado ese techo aparezcan timeouts en vez de respuestas rápidas con datos corruptos.

Resultados

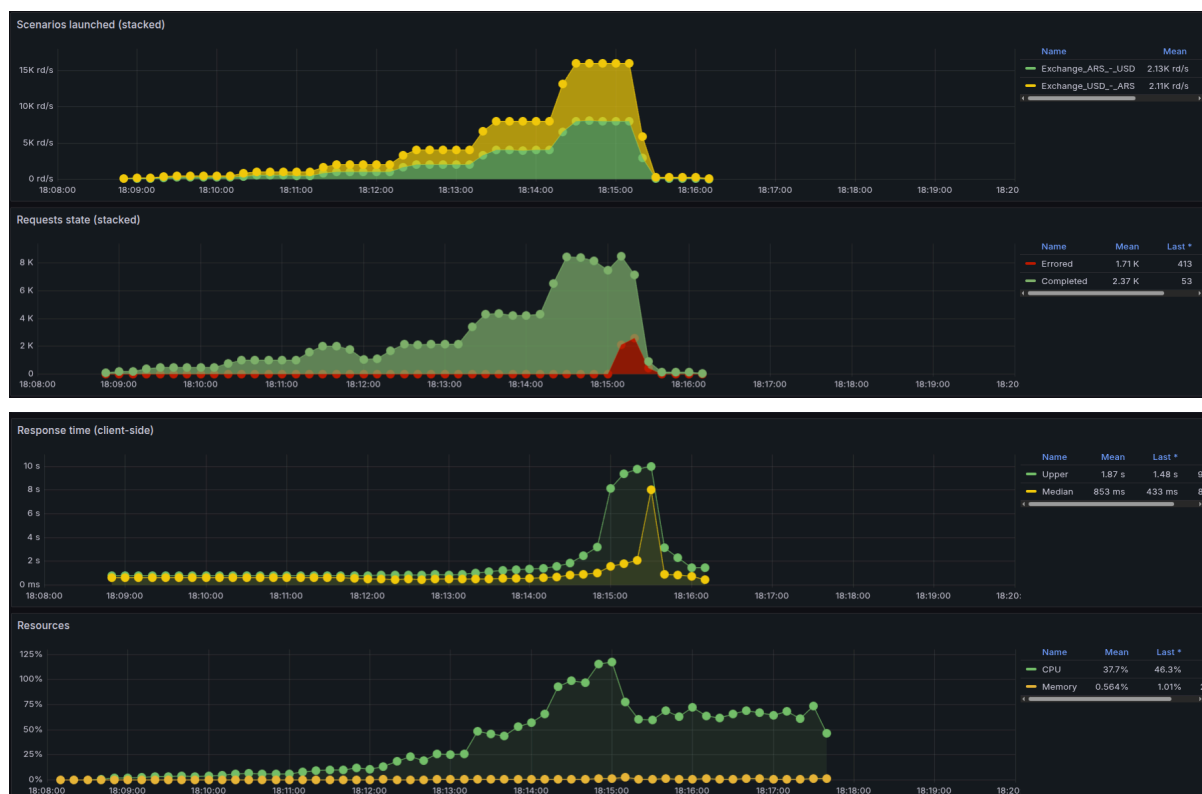
Carga normal: 390 requests, 0 fallos, media 596-610 ms, p95 ~728-743 ms, p99 ~743-773 ms. Idéntico al piso estructural que ya tenía.



Estrés:

Métrica	Valor
Requests totales	191.100
200 OK	106.977
500 (rechazo de negocio)	78.991
502	9
ETIMEDOUT	5.123 (2,7% vusers.failed)
p95/p99/máx global	2.231/3.328/9.934 ms

Por escalón, la latencia se mantiene en el piso de ~600 ms hasta el escalón de 400 req/s, y recién a partir de 800-1600 req/s crece fuerte (p50 1.135 ms, p95 2.180 ms, máx 9.385 ms en el escalón de 1600), con la CPU de la API llegando a 1,00 core (el límite del contenedor son 2). Los ETIMEDOUT aparecen exclusivamente en el escalón de 1600 req/s y en la recuperación.



Se revisó /log al terminar la corrida: de 187.602 exchanges procesados, exactamente 79.319 fallaron por "Not enough funds on counter currency account", cero por excepciones reales. Es decir, casi todo el bloque de 500 no es un fallo del sistema, sino el rechazo de negocio esperado devuelto con un código HTTP discutible (debería ser 409/422, no 500). Los saldos finales después de las 191.100 requests fueron:

ARS	USD	EUR	BRL
11.240	139.393,54	40.000	60.000

Ningún saldo quedó negativo ni inconsistente.

Qué se puede afirmar

- A nivel de código, la ventana de carrera identificada en modular (lectura de saldo en `if (counterAccount.balance >= counterAmount)`, seguida de dos `await transfer()` de 200-400 ms cada uno, y recién ahí la escritura con `adjustAccountBalance`) queda eliminada por construcción en transactions: el chequeo y las dos escrituras pasan a ejecutarse dentro de la misma sección crítica (`withAccountsLock`), antes de cualquier `await transfer`. No es una mejora probabilística ("es menos probable que pase"), es estructural: mientras un exchange tiene el lock de un par de cuentas, ningún otro exchange sobre ese mismo par puede entrar a su `fn()` hasta que el primero libere.
- El lock cierra la ventana de lost update que existía en modular: bajo 191.100 requests concurrentes contra el mismo par de cuentas, no se observó ningún saldo

negativo ni inconsistente. Esto es evidencia directa de la corrección del mecanismo, no una inferencia.

- Bajo carga normal, el lock no introduce costo medible: la latencia y la tasa de fallos son indistinguibles de modular/main.
- Bajo estrés, el lock introduce un techo de capacidad claro y explicable sobre un mismo par de cuentas (del orden de $\sim 1,6$ exchanges/segundo sobre ese par, dado que cada operación serializada tarda ~ 600 ms), y ese techo es el origen directo de los timeouts que aparecen a partir de 1600 req/s.
- A diferencia de la batería completa de modular —donde ningún endpoint recuperaba latencia normal en la ventana de recovery—, acá POST /exchange sí vuelve a un p50 cercano al piso normal dentro de la misma ventana de recovery (aunque arrastrando timeouts los primeros segundos).

Qué no se puede afirmar / matices

- No es comparable número a número contra la fila "POST /exchange" del informe de modular, porque no está confirmado que ambas corridas hayan usado el mismo patrón de acceso a cuentas (acá se fuerza a propósito a que todas las requests compitan por el mismo par; no está documentado si el escenario de modular hacía lo mismo). Sí es comparable la conclusión estructural (dónde está el techo, qué tipo de fallas aparecen).
- El resultado de recovery favorable es de una sola corrida, un solo endpoint: no alcanza para afirmar que el problema de recovery observado en modular esté resuelto en general.
- El techo de $\sim 1,6$ req/s es el techo para un mismo par de cuentas. En un escenario más realista, con miles de pares de cuentas operando en paralelo (cada uno con su propio lock independiente), este techo no debería notarse de la misma forma — pero esto no se probó en esta sesión.
- La tabla por escalón es una reconstrucción aproximada a partir de series de Graphite con resolución de 10 s, no un conteo en vivo por fase; coincide con el resumen final de Artillery dentro de $\sim 2\%$, pero los valores por escalón individual pueden tener más ruido que un conteo exacto.
- No se corrió la batería completa de 6 endpoints en esta rama, solo POST /exchange (único endpoint tocado por el cambio). Que los demás endpoints no se vean afectados es una inferencia por lectura de código (no comparten estado ni lógica con exchange()), no una medición.

Trade-off

La táctica mejora consistencia/corrección a costa de throughput bajo contención alta sobre el mismo recurso: se eligió serializar el acceso al par de cuentas en vez de usar locking optimista o particionar el estado de otra forma. Bajo carga normal el costo es nulo porque casi no hay contención real; el costo solo se paga cuando muchas requests compiten por las mismas dos cuentas al mismo tiempo, momento en el que el sistema pasa de "estado potencialmente incorrecto pero rápido" a "estado siempre correcto pero con cola y, eventualmente, timeouts".

Es un trade-off consciente y localizado: no se sacrificó throughput global del servicio, sino el throughput máximo alcanzable sobre un par de cuentas específicas bajo contención extrema.

Conclusión del caso

El lock resuelve el problema que se propuso resolver — la condición de carrera de modular — sin efectos negativos medibles bajo carga normal, y con un costo de throughput que solo se manifiesta bajo un escenario de estrés diseñado específicamente para maximizar la contención sobre un mismo par de cuentas. El diseño (mutex por cuenta con orden determinístico para evitar deadlocks) es apropiado para el problema puntual, aunque no fue validado a la escala de múltiples pares de cuentas en paralelo, que sería el siguiente paso antes de extender esta conclusión al sistema completo.

Táctica 3: Concurrencia

Motivación

“Transactions” (rama previa) ya había resuelto la condición de carrera de modular metiendo el chequeo de saldo + el ajuste de los dos balances internos dentro de un único `withAccountsLock`. Esa parte quedaba correcta. El problema que quedaba pendiente era de latencia, no de corrección: una vez reservados los fondos dentro del lock, las dos transferencias externas simuladas se seguían ejecutando en secuencia (`app/services/exchange.js`, versión `transactions`):

```
await transfer(clientBaseAccountId, baseAccount.id, baseAmount);
await transfer(counterAccount.id, clientCounterAccountId, counterAmount);
```

Cada `transfer()` es un stub que tarda 200-400 ms. Al esperarlas una tras otra, cada `/exchange` exitoso pagaba el costo de las dos (~600-800 ms de piso), pese a que las dos transferencias son independientes entre sí una vez que el dinero ya fue reservado internamente — no hay ninguna razón de negocio para que una espere a la otra.

Esto es visible primero en el código, y después se confirma con evidencia empírica: la carga normal de `transactions` mediaba ~600 ms por request, calzando casi exacto con la suma de dos tramos de 200-400 ms.

Táctica aplicada

Paralelizar operaciones independientes en `app/services/exchange.js`.

Cambios:

- Las dos `transfer()` se lanzan juntas con `Promise.allSettled([...])` en vez de `await` una tras otra.

- Como consecuencia, aparecen combinaciones de resultado que en transactions no podían darse (ahí, si la primera fallaba, la segunda ni se intentaba). Ahora se manejan cuatro casos explícitos:
- inflowOk && outflowOk → éxito.
- inflowOk && !outflowOk → se cobró al cliente pero no se le pagó: se revierte el cobro y se libera la reserva.
- !inflowOk && outflowOk → se le pagó al cliente sin cobrarle: se recupera el pago y se libera la reserva.
- !inflowOk && !outflowOk → ninguna de las dos: solo se libera la reserva.
- Lo que no cambió: el withAccountsLock sobre [baseAccount.id, counterAccount.id] que protege el chequeo-de-saldo + ajuste de balances, y releaseReservation(). La garantía de que el saldo nunca queda negativo sigue viniendo de esa sección crítica, no de la paralelización — esta táctica ataca la latencia, no la condición de carrera (esa ya estaba resuelta).

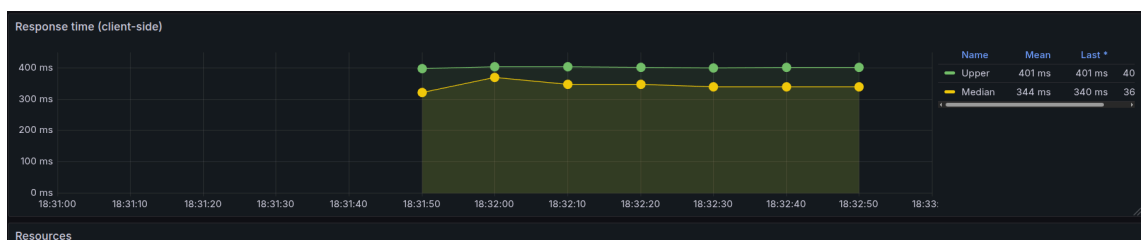
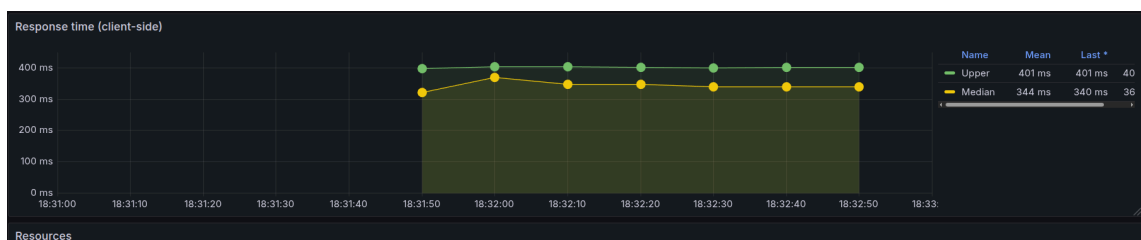
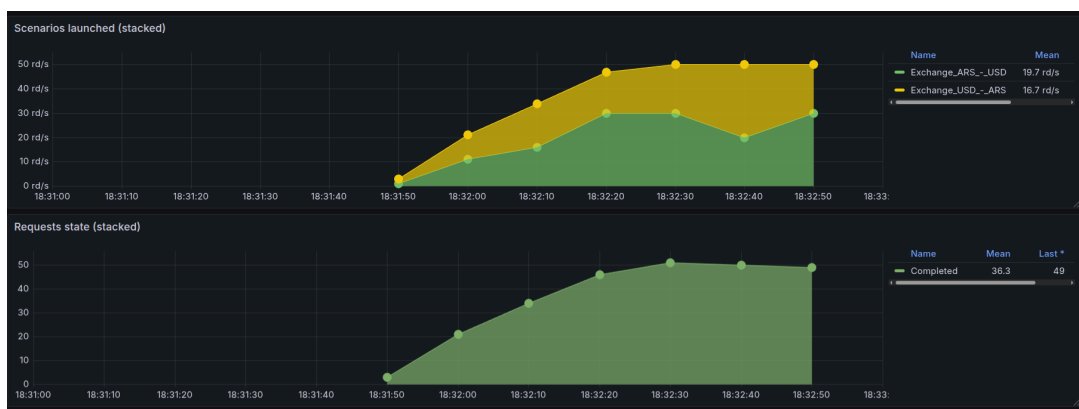
Predicción

- El piso de latencia de un exchange exitoso debería bajar de $t_1 + t_2$ (~600-800 ms) a $\max(t_1, t_2)$ (~200-400 ms) — aproximadamente a la mitad.
- La corrección (balance nunca negativo) no debería cambiar: el lock de reserva sigue intacto.
- Bajo estrés extremo, al reducirse el tiempo que cada request retiene el lock, el sistema debería poder sostener más throughput antes de que la cola de locks empiece a cruzar el timeout — es decir, se esperaba un colapso más tarde o más leve que en “transactions”, no necesariamente su desaparición.
- Uso de CPU: sin cambios esperados, porque el trabajo total (dos transfers) es el mismo; solo cambia si se hace en paralelo o en serie.

Resultados

Carga normal:

390 requests, 0 fallos. Media 335,5 ms, mediana 347,3 ms, p95/p99 399,5 ms, mínimo 210 ms, máximo 401 ms.



Resources

Overdraft:

Ráfaga de 30 s @ 20 req/s drenando a propósito la cuenta USD (60.000 USD de saldo real de seed), para forzar exactamente la zona donde viviría un bug de condición de carrera si el lock no sostuviera la invariante.

600 requests, 31 200 OK y 569 500 (rechazo de negocio, "Not enough funds on counter currency account"), 0 timeouts. Balance final verificado con GET /accounts: USD terminó en 1.095, nunca negativo, pese a 600 intentos concurrentes de drenarlo.

31 operaciones exitosas coincide con el cálculo a mano: con tasa ARS→USD de 0,00066 y baseAmount 1.750.000 ARS por request, cada exchange exitoso consume ≈ 1.155 USD; con 60.000 USD de partida el techo teórico es ≈ 52 operaciones — el lock cortó antes (31) porque varias requests concurrentes compiten por la misma ventana de saldo restante y sólo una gana el lock cuando ya no alcanza para dos.

Los 200 OK promedian ~ 320 ms (costo real de transferir); los 500 promedian < 1 ms (el chequeo de saldo falla dentro del lock, antes de llamar a ningún transfer()) — el camino de rechazo es barato.



Estrés:

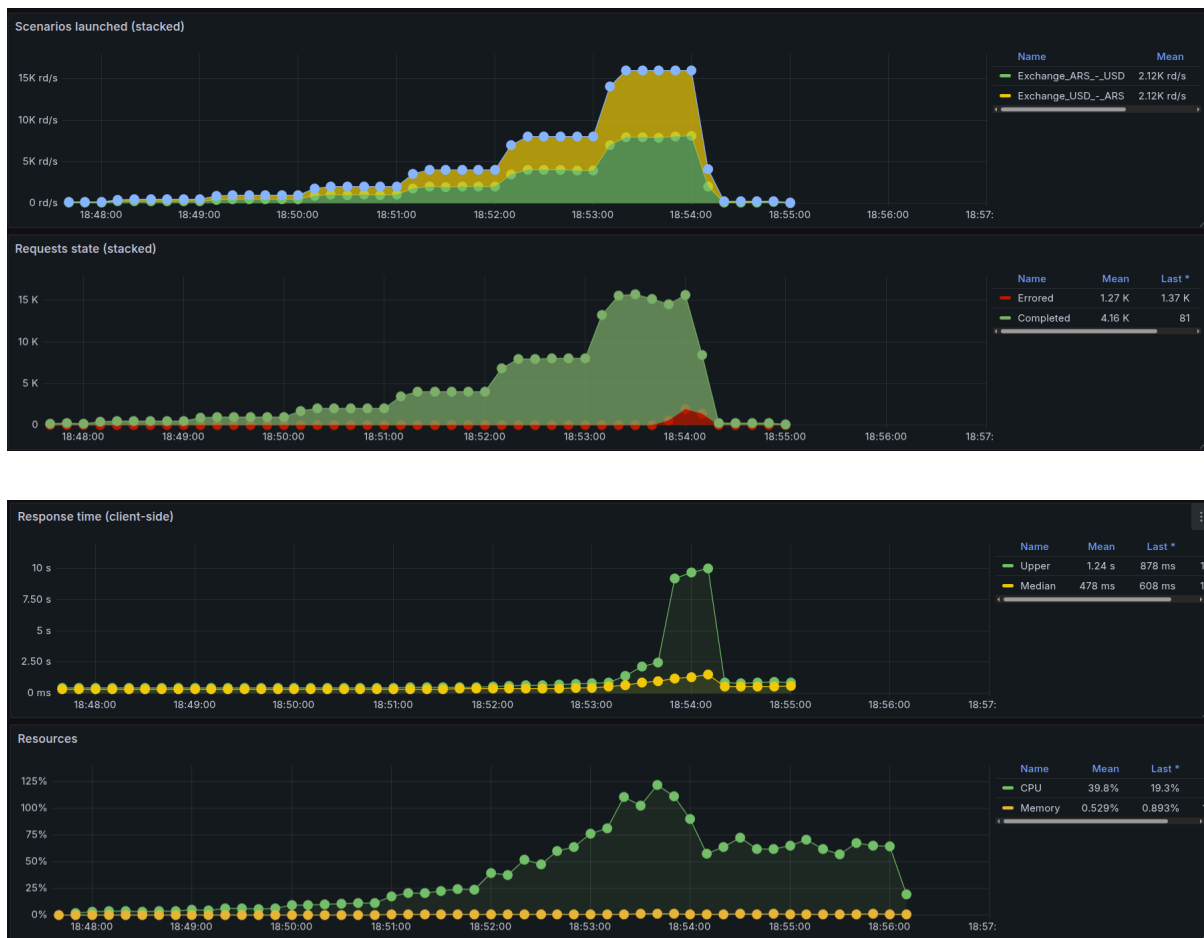
Step-load completo (Warm 20 → 50 → 100 → 200 → 400 → 800 → 1600 → Recovery 25 req/s), pool 300, timeout HTTP 10 s, cuentas precargadas a 1 billón por el before propio del yaml (acá no hay rechazos por fondos insuficientes, el foco es throughput/latencia bajo carga).

Métrica	Valor
Requests totales	191.100
200 OK	187.289 (98,0 %)
502	6
ETIMEDOUT	3.805 (2,0 %)
p95/p99/máx	1.620/4.231/9.870 ms

Por escalón (reconstrucción aproximada desde series de Graphite, resolución 10 s):

Escalón	200 (aprox)	502	Timeouts	p50/p95/p99	Máx (ms)	CPU api / nginx (cores)
Warm up (20 rps)	540	0	0	345/400/400	403	0,02/0,01
Step 50 rps	2.915	0	0	345/400/400	406	0,04/0,01
Step 100 rps	5.860	0	0	343/400/400	422	0,06/0,02
Step 200 rps	11.720	0	0	344/400/404	444	0,12/0,03
Step 400 rps	23.450	0	0	351/416/456	510	0,25/0,05
Step 800 rps	46.780	6	0	402/601/641	791	0,56/0,10
Step 1600 rps	87.610	0	2.437	913/1.911/3.010	9.704	1,03/0,78
Recovery (25 rps)	8.160	0	1.368	619/1.981/2.256	10.000	0,64/0,01

El colapso arranca en el mismo escalón que en transactions (1600 req/s), pero con menos severidad: 2,0 % de timeouts totales contra el 2,7 % de transactions (~26 % menos). La recuperación sí vuelve al piso normal dentro de la misma ventana de 60 s (p50 arranca en 1.495 ms tras el pico y baja a 550-608 ms).



Qué se puede afirmar

- La paralelización de las dos transferencias reduce a la mitad el piso de latencia en carga normal (~ 600 ms $\rightarrow$ ~ 335 ms), consistente uno a uno con la predicción $\max(t_1, t_2)$ vs $t_1 + t_2$ y con el máximo observado (401 ms, que calza con un solo tramo del rango 200-400 ms del stub, no con la suma de dos).
- La invariante de corrección de transactions se mantiene. El overdraft dedicado (600 intentos concurrentes de drenar la misma cuenta) confirma que el balance nunca queda negativo — el lock de reserva sigue siendo la pieza que garantiza esto y no se vio afectada por el cambio.
- Bajo estrés extremo, el sistema tolera más carga antes de colapsar: la tasa de timeouts baja de 2,68 % a 1,99 % en el mismo step-load, y el punto de arranque del colapso (1600 req/s) es el mismo pero la severidad es menor.
- El costo de CPU es equivalente entre ambas ramas (1,00 vs 1,03 cores en el pico): la mejora es de tiempo de espera (I/O simulado), no de trabajo de cómputo ahorrado.

Qué no se puede afirmar / matices

- El colapso a 1600 req/s no desaparece, solo se atenúa. Esta táctica no resuelve el techo de capacidad del sistema, lo corre un poco más lejos.
- Solo se corrió la batería de /exchange; no se repitió la batería completa de los otros 5 endpoints en esta rama, dado que el cambio no los toca.

Trade-off

- A favor: mejora de latencia percibida (mitad del piso) y de resiliencia bajo estrés (menos timeouts), sin costo adicional de CPU ni cambio en el mecanismo de corrección existente (el lock).
- En contra: la superficie de código creció — de un solo camino feliz secuencial a cuatro combinaciones de resultado (Promise.allSettled + 4 ramas), lo cual es más difícil de razonar. Si en el futuro transfer deja de ser un stub que siempre resuelve true y empieza a fallar de verdad, esta complejidad recién ahí se paga (o no) — hoy es deuda de testing, no de diseño.
- No hay trade-off de corrección: la garantía de saldo no-negativo depende del lock, que no se tocó.

Conclusión del caso

Mejora, no regresión. Paralelizar las dos transferencias externas reduce a la mitad el piso de latencia bajo carga normal y reduce la tasa de timeouts bajo el mismo step-load agresivo que hace colapsar a transactions, sin costo aparente de CPU ni de corrección — el overdraft dedicado confirma que el balance nunca queda negativo pese a la paralelización. El punto abierto es de testing, no de diseño: las rutas de reconciliación que esta paralelización necesita (cobrado-sin-pagar / pagado-sin-cobrar) están escritas pero nunca se disparan con el transfer() stub actual (siempre resolve(true)). Para tener confianza real ahí haría falta introducir fallos simulados en transfer() y un escenario dedicado a forzarlos.

Táctica 4: Redis y load balancer

Motivación

La táctica anterior resolvió la condición de carrera de /exchange con un lock en memoria del proceso, y cuentas, tasas y log vivían en variables del mismo proceso. Eso funciona con una sola instancia, pero impide escalar horizontalmente: una segunda instancia no tiene forma de enterarse de lo que hizo la primera, y cada una aprobaría operaciones contra su propia copia del saldo.

Táctica aplicada

Se movió el estado de las cuentas a Redis, como una única key (accounts) con el JSON de las cuatro cuentas. La exclusión mutua de /exchange pasó del Map en memoria a dos scripts Lua, que Redis ejecuta de forma atómica: uno chequea el saldo y ajusta ambas cuentas, y el otro revierte la reserva si las transferencias fallan. Las tasas y el log se cargan desde Redis al iniciar, pero siguen residiendo en la memoria de cada proceso, que los vuelca completos a Redis de forma periódica (el log cada 1 s, las tasas cada 5 s). Sobre esta base se probó escalar la api a 3 réplicas detrás de Nginx como balanceador round-robin.

Predicción

Con el chequeo y ajuste de saldos resuelto de forma atómica en Redis, la consistencia de los saldos debería sostenerse con cualquier cantidad de réplicas.

Resultados

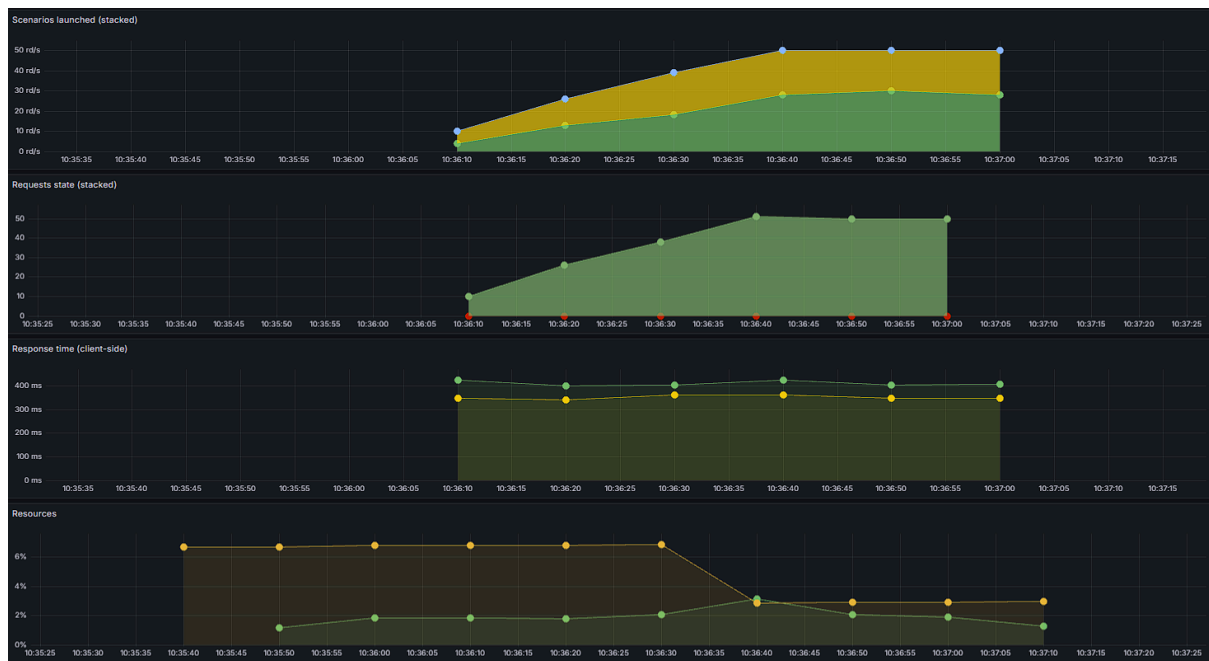
Se corrieron los mismos tres escenarios con 1 y con 3 réplicas, partiendo en ambos casos de un log vacío. Con 3 réplicas se verificó que el tráfico se repartiera en partes iguales entre las tres instancias.

Carga normal. 390 requests, 0 fallos en ambos casos. Media de 337 ms con 1 réplica y 336 ms con 3; p95 de 400 ms y máximo de 404 ms en las dos. Es el mismo piso que en concurrency (~335 ms): a esta carga, el acceso a Redis no agrega latencia medible.

Carga normal, 1 réplica

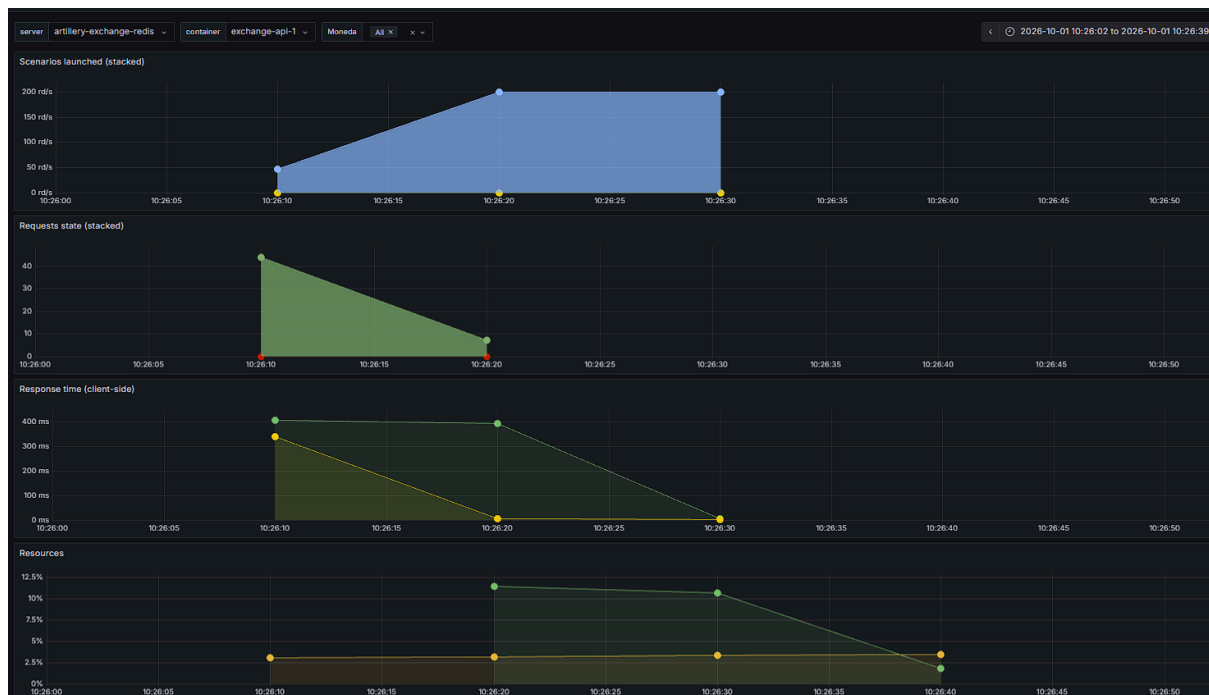


Carga normal, 3 réplicas

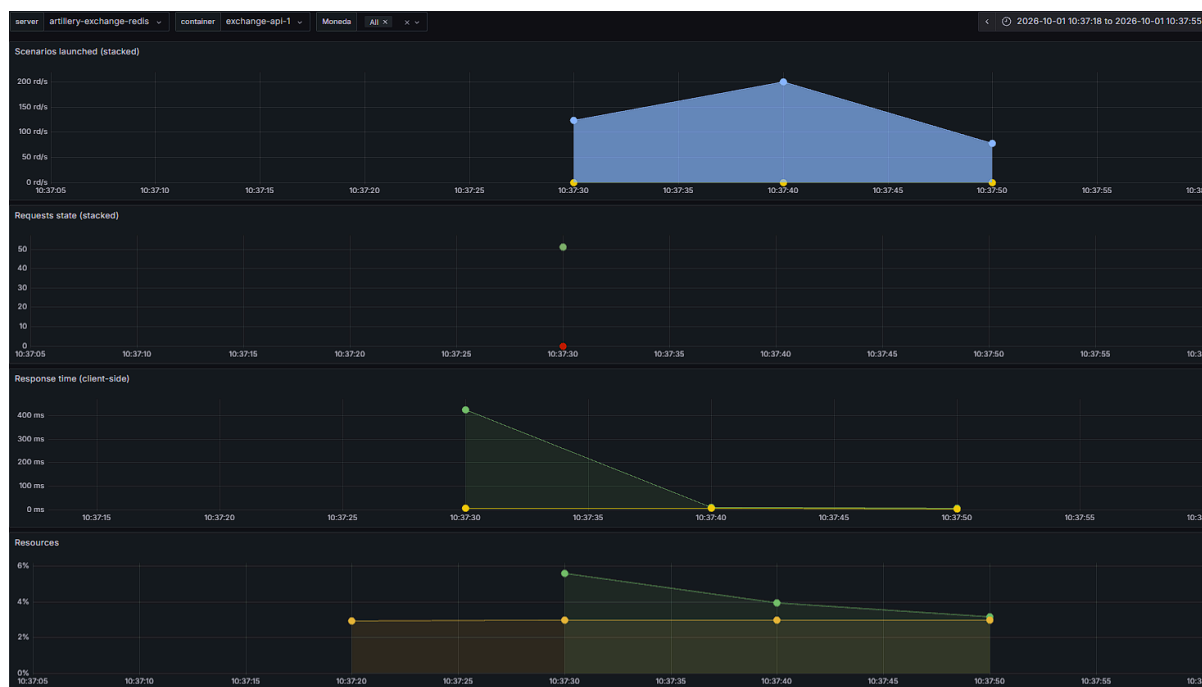


Overdraft. Ráfaga de 30 s a 20 req/s intentando drenar la cuenta USD, con 60.000 USD de saldo inicial. Con 1 y con 3 réplicas el resultado fue idéntico: 51 operaciones aceptadas, 549 rechazadas por fondos insuficientes y saldo final de 202,5 USD. Coincide con el cálculo a mano: cada operación consume 1.172,5 USD ($1.750.000 \text{ ARS}$ a tasa 0,00067), y $60.000 / 1.172,5 = 51,2$.

Overdraft, 1 réplica



Overdraft, 3 réplicas



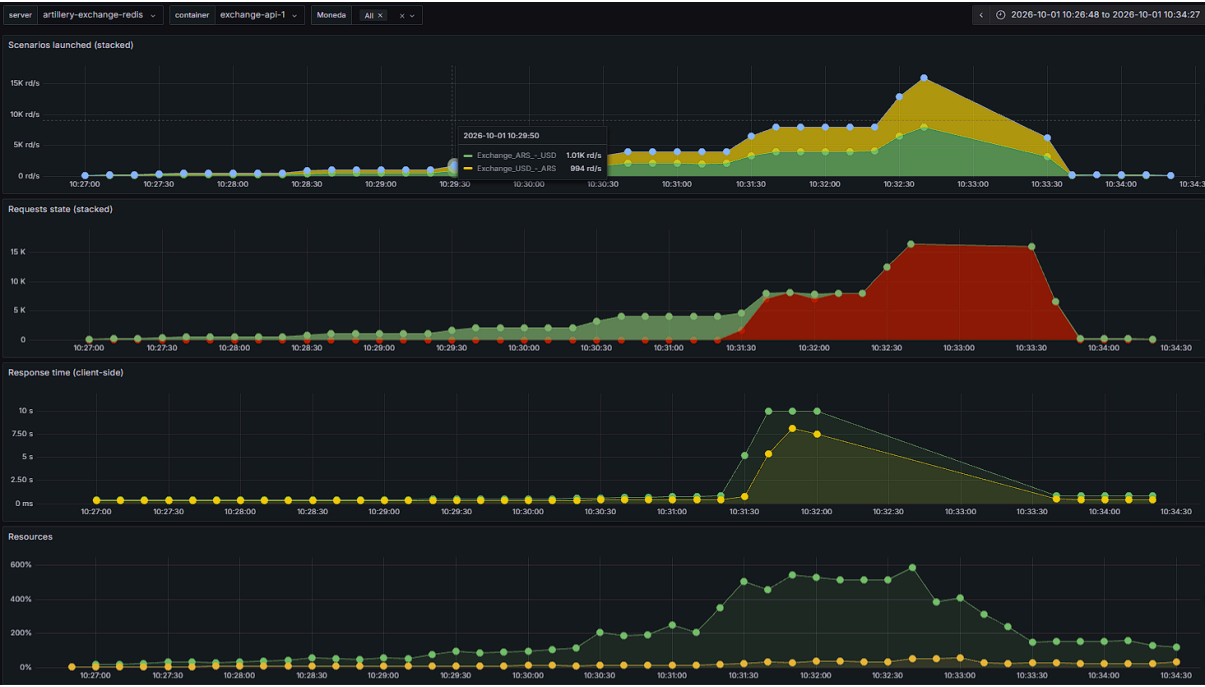
Estrés. Escalones de 60 s a 50, 100, 200, 400, 800 y 1600 req/s, 191.100 requests en total.
Performance bajo estrés:

Configuración	exchange carga normal (mean)	% éxito en estrés	Colapso empieza en
original (sin Redis)	~600 ms	58,5%	400-800 rps
concurrency (sin Redis)	~335 ms	98,0%	1600 rps
redis (1 réplica)	~335ms	26,2 %	800 rps
redis (3 réplicas)	~335ms	32,1%	800 rps

Hasta el escalón de 400 req/s ambas configuraciones responden sin fallos (p99 de 686 ms con 1 réplica y 529 ms con 3). En el escalón de 800 req/s las dos colapsan: la mediana pasa de ~350 ms a más de 7 s y aparecen timeouts y resets de conexión. En 1600 req/s prácticamente no se completa ninguna operación. Con 1 réplica hubo 60.152 timeouts y 66.024 resets; con 3 réplicas, 27.780 timeouts y 93.575 resets.

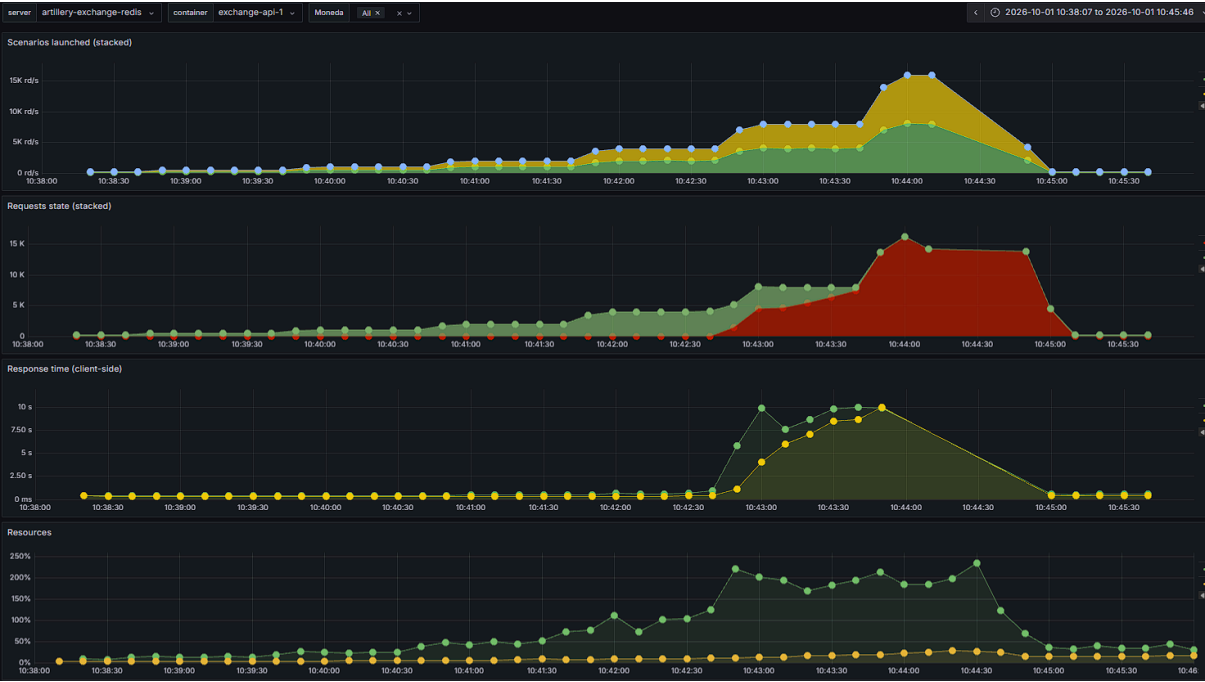
El uso de CPU muestra dónde está el límite: Redis llegó a 1,85 cores con 1 réplica y a 1,33 con 3, mientras que la api llegó a 1,17 cores con una instancia y a 0,47 por instancia con tres.

Estrés, 1 réplica

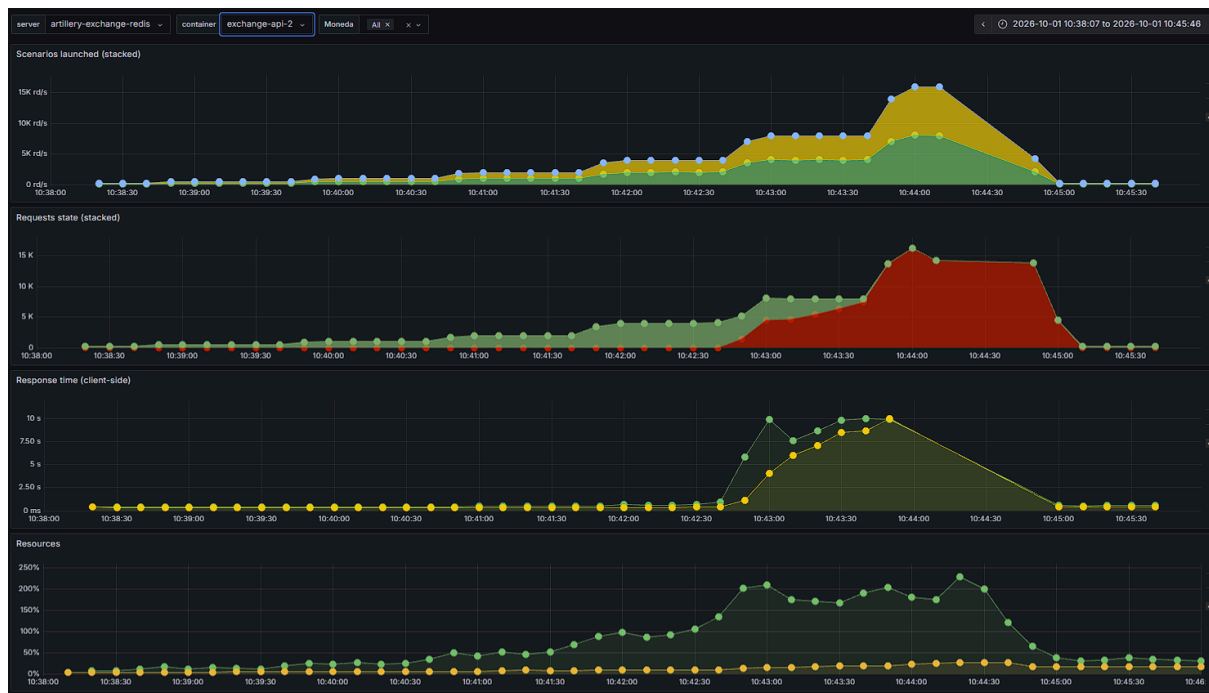


Estrés, 3 réplicas

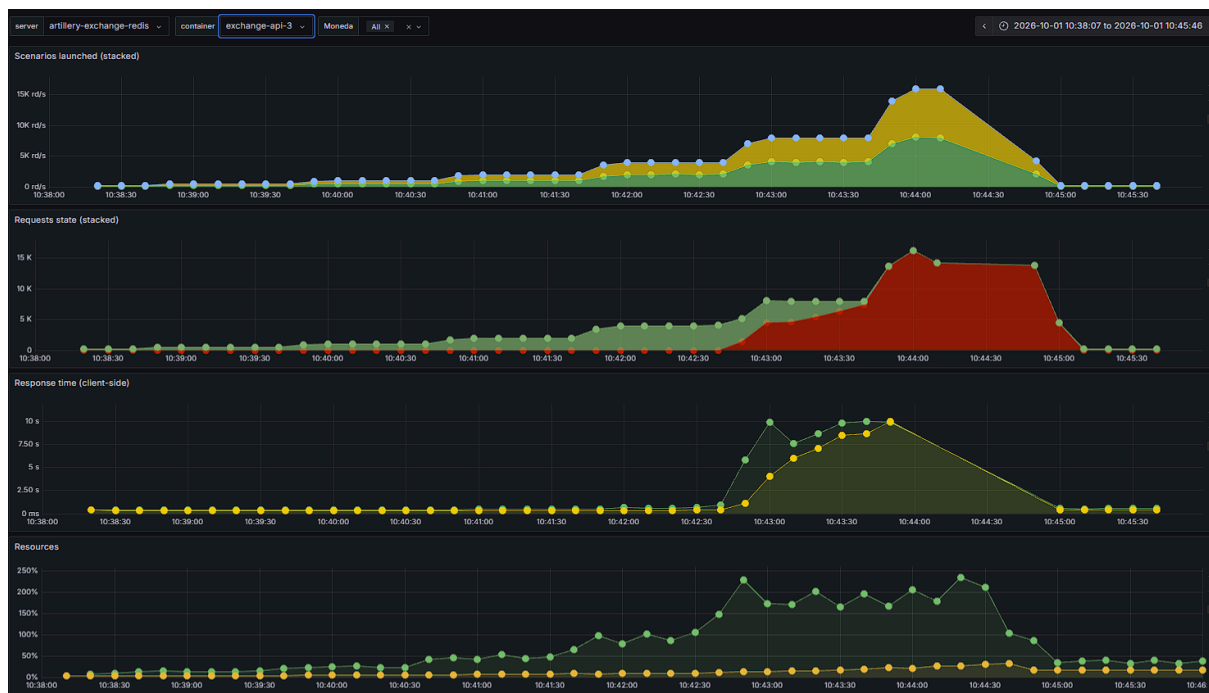
- Replica 1



- Réplica 2



• Réplica 3



Qué se puede afirmar

La consistencia de los saldos se sostiene entre réplicas. El overdraft dio exactamente el mismo resultado con 1 y con 3 instancias, y coincide con el máximo teórico. Con el lock en memoria de la táctica anterior, tres procesos habrían aprobado operaciones cada uno contra su propia copia del saldo.

Bajo carga normal, externalizar el estado no tiene costo medible de latencia.

Bajo estrés, la capacidad empeoró respecto de concurrency: el colapso se adelantó de 1600 a 800 req/s y el porcentaje de éxito cayó del 98 % a alrededor del 30 %.

Agregar réplicas no cambió el punto de colapso. Las tres instancias de api quedaron lejos de saturarse (0,47 cores cada una sobre un límite de 2), lo que indica que el cuello de botella no está en la api.

Trade-off

Gana	Pierde / empeora
Consistencia de saldos entre instancias	Performance bajo estrés
Escalabilidad	Consistencia del log y las tasas con más de una réplica
Persistencia	Simplicidad

Conclusión del caso

La predicción se cumplió para la consistencia de saldos y falló para la performance. Externalizar el estado es condición necesaria para escalar horizontalmente, pero no suficiente: mientras el modelo de datos concentre las escrituras en una única key y el log se reescribe entero cada segundo, sumar instancias no agrega capacidad.

La configuración entregada en docker-compose.yml utiliza una única instancia de api. El escalado a 3 réplicas se realizó solo como experimento para esta táctica, porque el log y las tasas todavía no están compartidos entre instancias. Para escalar correctamente habría que mover también esos datos a estructuras compartidas en Redis.

Components & Connectors

Se agrega el componente Redis (antes el estado vivía en memoria de api y en archivos .json locales), con un conector nuevo api↔Redis usado tanto para las lecturas simples (GET/JSON.parse) como para los dos scripts Lua atómicos.

El componente api deja de ser una instancia única y pasa a ser un conjunto de réplicas intercambiables detrás de un balanceador round-robin en Nginx (conector cliente→Nginx→api ya no apunta a un contenedor fijo, sino a un upstream con N IPs resueltas por Docker DNS).



Táctica 5: Log paginado y métricas de volumen y netos

Motivación

Suponiendo un uso normal de sistema, por cada transacción que se hace el log se agranda, esto lleva a que pasado un tiempo el costo de hacer GET /log, debido a su tamaño, sea muy costoso respecto a recursos generando mayor latencia y peor performance.

Táctica aplicada

Se decidió limitar el tamaño de la respuesta, agregando un parámetro a la query ?limit= (entero entre 1 y 500, default 100) a GET /log que indica que le devolverá las últimas “n”entradas en vez del arreglo completo.

Además, se agregaron métricas de volumen y netos por cada moneda operada. Este es un requisito obligatorio que no tiene efectos más que de mejorar la observabilidad.

Predicción

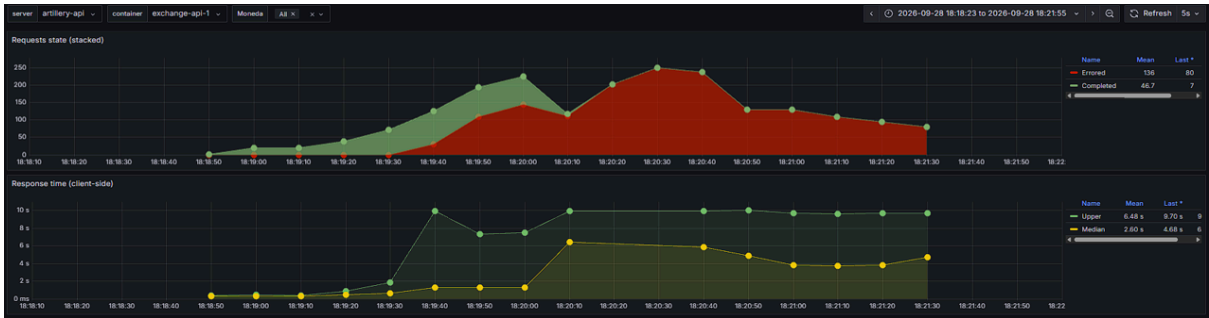
Si limitamos el payload de GET /log a un tamaño fijo, independiente del tamaño total acumulado del log, debería eliminar (o reducir drásticamente) la degradación que un log grande provoca tanto en carga normal como en estrés.

Resultados

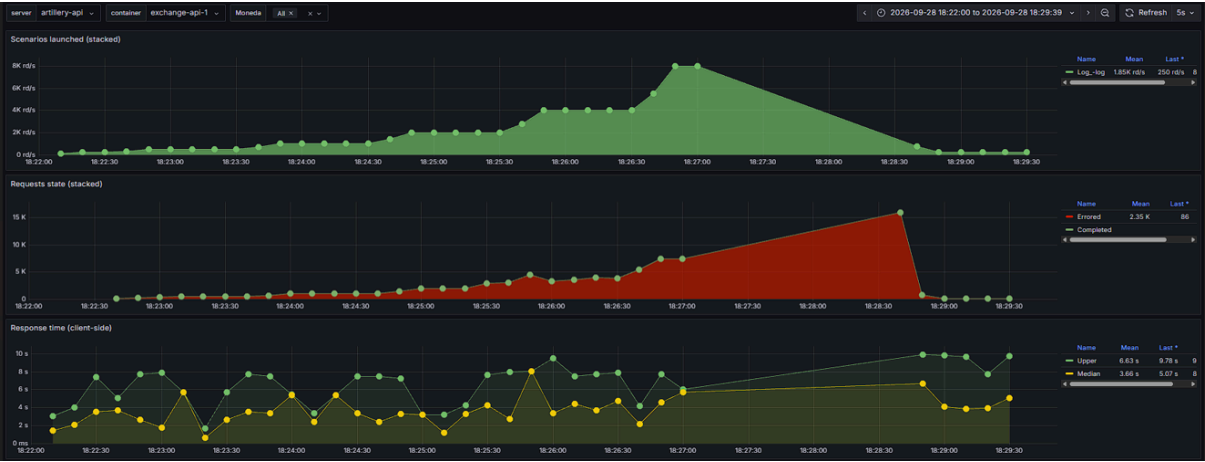
Log sembrado una única vez con 89.036 entradas, mismo volumen que el utilizado para las pruebas de la táctica anterior (Redis), siempre con 1 sola réplica para no mezclar el efecto de tener tres réplicas con la paginación del log

Métrica	redis — sin paginar	log-pagination — con paginación
Éxito en carga (log.yaml)	18,2%	100%
Éxito en estrés (log-stress.yaml)	0%	25–65%
p99 en carga	5.827 ms	46 ms

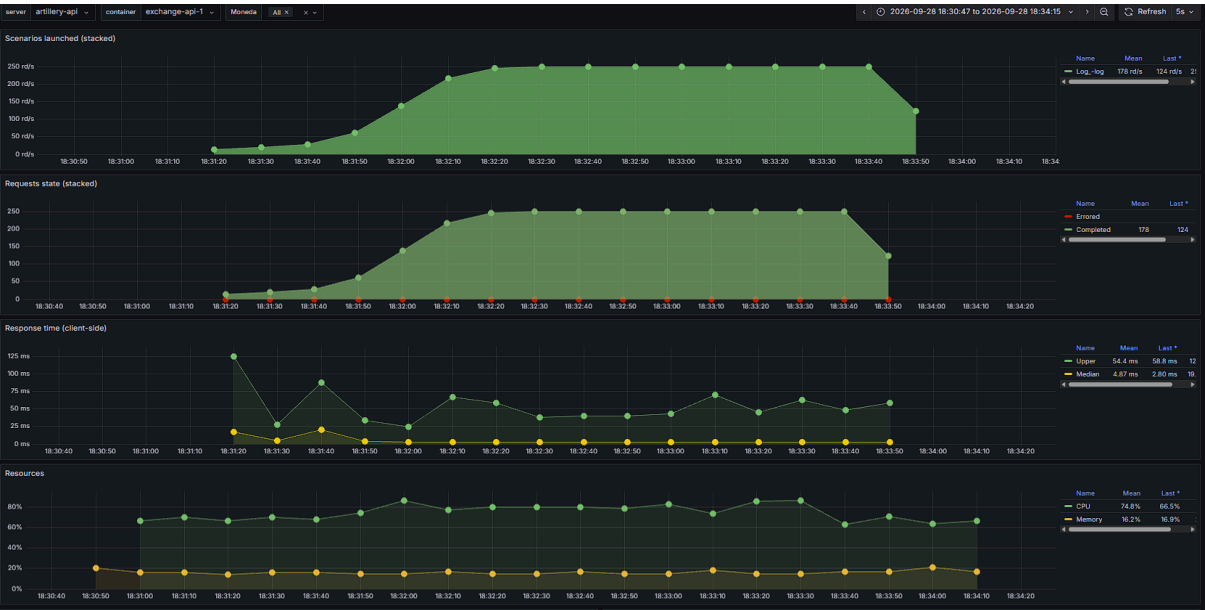
Redis sin paginar en carga



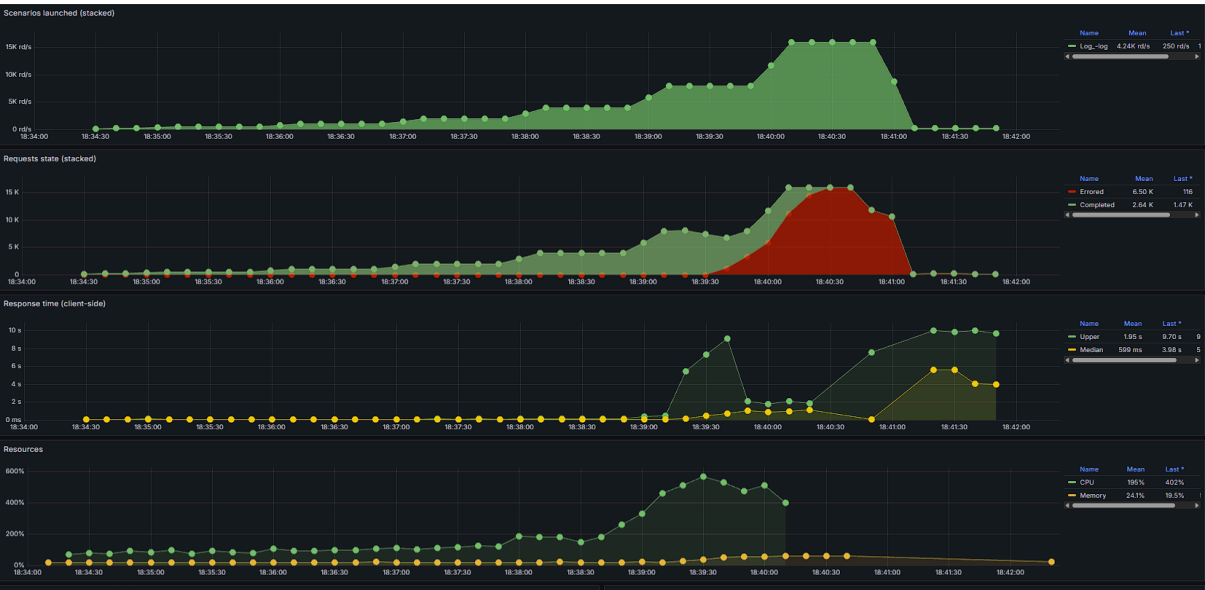
Redis sin paginar en estrés



Log carga



Log estrés



Qué se puede afirmar

Si suponemos que se pide un log luego de que el sistema tenga un ciclo de vida entonces paginar tiene el impacto más grande y más consistente.

Trade-off

Gana	Pierde / empeora
Performance	Usabilidad
Observabilidad	

Conclusión del caso

Predicción confirmada con el margen más amplio de todos los casos medidos.

Táctica 6: Cache y Health

Motivación

GET /log era, de los seis endpoints medidos, el que colapsaba con menos carga: aun aplicando la limitación la respuesta en la táctica anterior, el servicio empezaba a fallar mucho antes que el resto. Como todas las responses son iguales sin importar quien hace la reques se decidió aplicarle una cache.

Táctica aplicada

Se agrega una cache en Nginx (proxy_cache_path) con TTL de 1 segundo sobre GET /log, más proxy_cache_lock (para que un pico de tráfico no dispare N requests idénticos al backend mientras la entrada está vacía) y proxy_cache_use_stale updating (sirve la copia vieja mientras se refresca, en vez de bloquear).

De forma secundaria, se agregó GET /health (ping a Redis con timeout de 500ms) — una mejora de observabilidad, no de esta táctica, pero forma parte del contenido de la rama.

Predicción

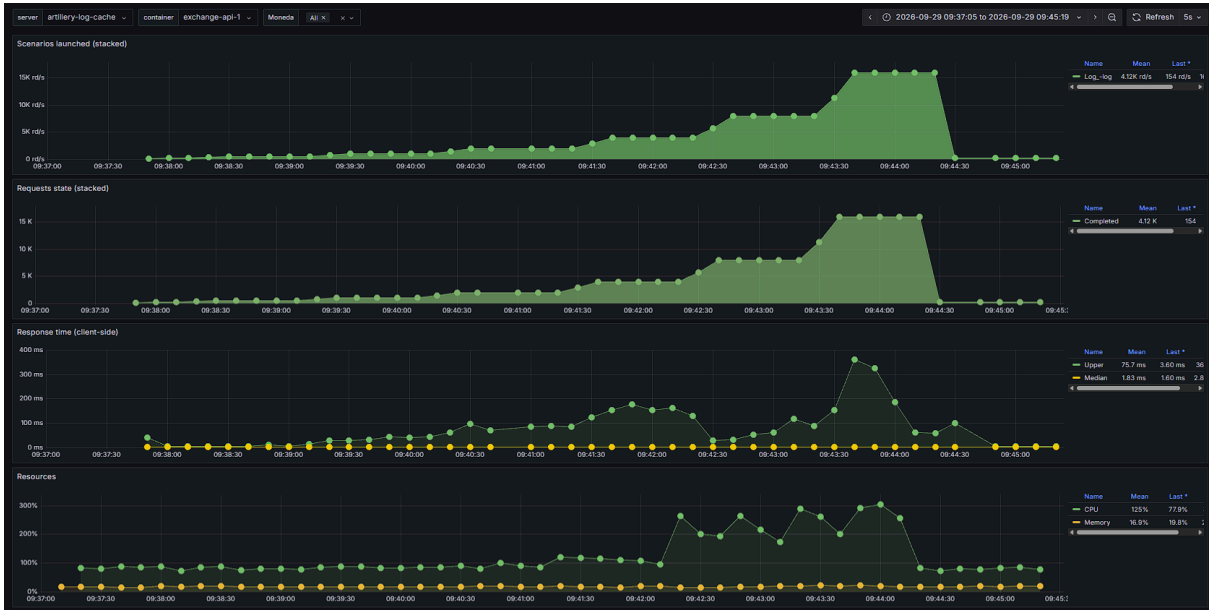
Al servir la mayoría del tráfico repetido desde la cache sin tocar el backend, el servicio debería sostener el mismo perfil de carga (hasta 1600 rps) sin colapsar, aunque cada request individual a /log siga siendo relativamente cara de calcular la primera vez.

Resultados

Las métricas evidencian un gran cambio respecto a agregar un cache que no tenerlo

Métrica	Sin cache	Con cache (1s)
Requests totales	190.650	191.100
Fallidas	105.838	0
ECONNRESET	97.972	0
ERR_SOCKET_TIMEOUT	7.866	0
Mean response time	1930,5 ms	1,4 ms
p95	5272,4 ms	2 ms
p99	9607,1 ms	19,1 ms

Con cache 1s (log-cache)



Qué se puede afirmar

El caché de 1s con lock elimina el colapso para tráfico 100% cacheable, con margen amplio (0 fallos donde antes fallaba el 55% de las requests).

Trade-off

Gana	Pierde / empeora
Disponibilidad, por menor carga	Consistencia (datos hasta 1s desactualizados)

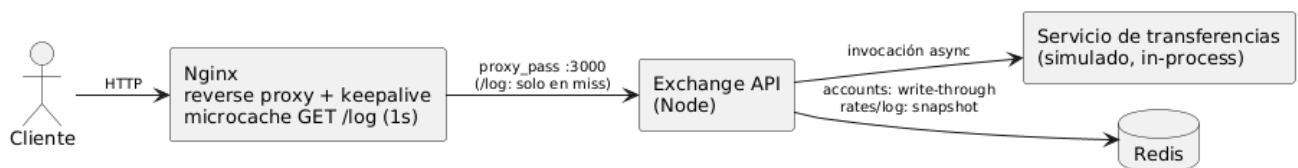
Performance	
Escalabilidad	
Observabilidad	

Conclusión del caso

Predicción confirmada con evidencia cuantitativa fuerte, sin matices que la contradigan. También se había evaluado la posibilidad de aplicar el mismo patrón a GET /rates pero esto podría generar un problema dado que la consistencia es débil y la fiabilidad no estaría garantizada. Pudiendo generar que un cliente obtenga un dato desactualizado, generando que una tasa vieja pueda afectar el resultado de un exchange.

Components & Connectors

Se agrega un componente de cache (zona log_cache) interpuesto en el conector Nginx→api para el path /log, un HIT nunca llega al componente api. También se agrega un conector nuevo y directo api→Redis para el ping de /health, por fuera del flujo de negocio habitual.



Táctica 7: Rate limiting y manage resource

Motivación

Ningún endpoint del sistema tenía freno de entrada: cualquier pico de carga se traduciría directo en saturación de Redis y de las conexiones de Node, con timeouts y 502 como único modo de falla. El objetivo era convertir ese colapso caótico en un rechazo controlado (429) que preserve el resto del tráfico.

Táctica aplicada

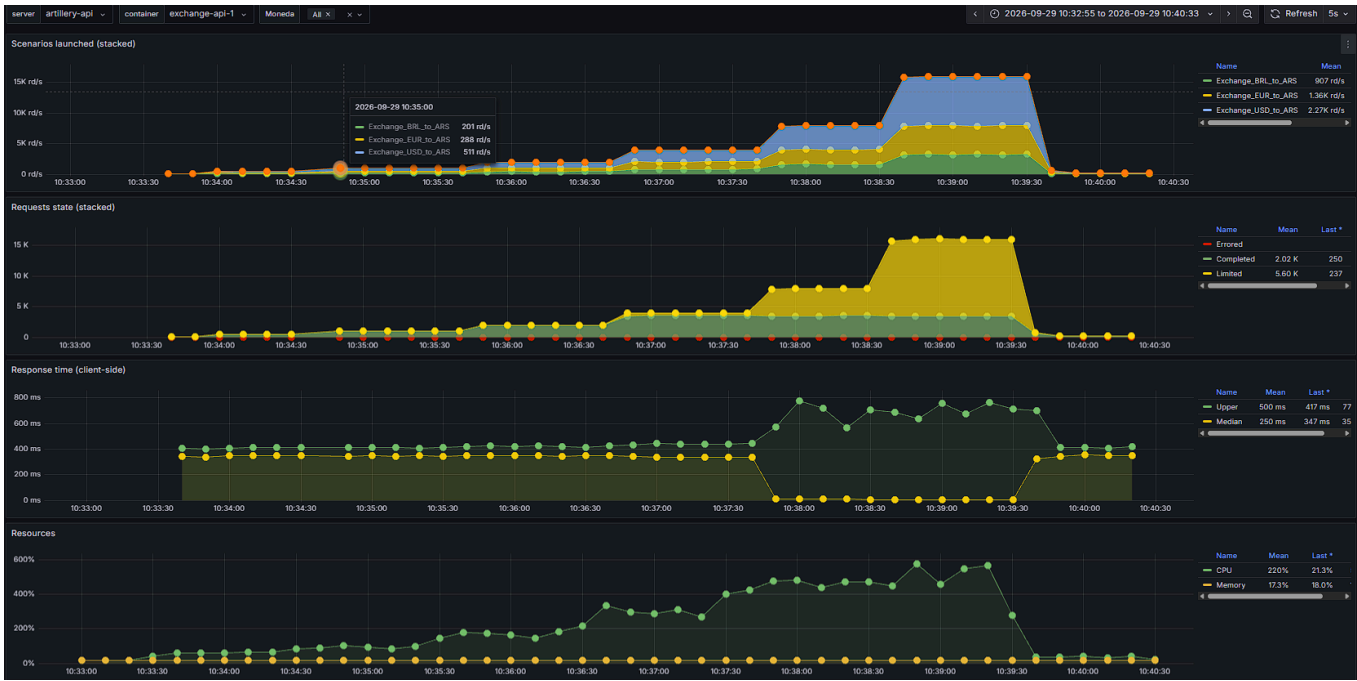
Bound Queue Sizes y Manage Resources aplicando un rate limiting. Se probaron dos variantes, la primera era aplicarlo dentro de la app pero no funcionó. La segunda lo aplico en Nginx, rechazando antes de abrir conexión hacia el backend.

Predicción

Agregar rate limiting en /exchange y /log debería hacer que el servicio, en vez de colapsar sin control bajo estrés, rechace con 429 de forma prolija y sostenga el resto del tráfico.

Resultados

Configuración	Fallos reales	Éxitos (200)
Sin protección	135.593 (71%)	50.221
Límite en Nginx (350 r/s)	0	85.358 (+105.291 con 429)



Qué se puede afirmar

La predicción original se cumplió, pero solo con la táctica aplicada en Nginx. Con Express solo, el resultado fue igual o peor que no tener nada.

Trade-off

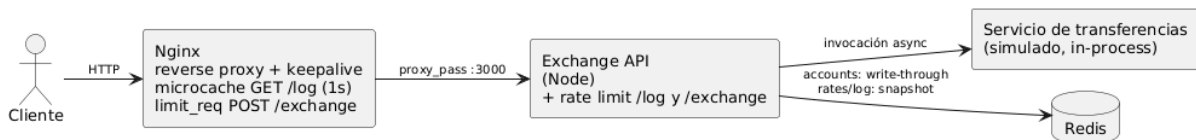
Gana	Pierde / empeora
Disponibilidad del servicio	Usabilidad
Performance	

Conclusión del caso

La predicción se sostuvo, pero no con la primera táctica intentada, y si cuando se la aplicó en Nginx. Demostrando que importa dónde se aplica ya que ponerlo en distintos lados puede no generar un cambio o incluso empeorarlo.

Components & Connectors

Nuevo componente leaky bucket (limit_req) en Nginx, interpuesto en el conector cliente→api para /exchange, que puede rechazar sin siquiera abrir conexión hacia el componente api. También un componente de rate limiting dentro de api (middleware Express) en el conector router→handler de /log y /exchange.



Táctica 8: Redis, partición del ledger de cuentas

Motivación

En la táctica anterior se aplicó rate limit, luego de hacerlo notamos que como funcionaba Redis en accounts.js generaba un cuello de botella ya que todas las transferencias comparten una única key de Redis (accounts) con un blob JSON de las 4 cuentas, que un script de Lua reescribe entero en cada operación.

Táctica aplicada

Lo que se hizo fue cambiar la forma en la que se particiona el estado compartido para bajar el costo por operación. Se reemplazó la key única por una key de Redis por cuenta, y el script de Lua de transferencia pasó a tomar y modificar solo las 2 keys involucradas en cada operación, en vez de reescribir el todo.

Predicción

Si bajamos el costo de cada operación entonces deberían subir los máximos, generando un mejor rendimiento medio.

Resultados

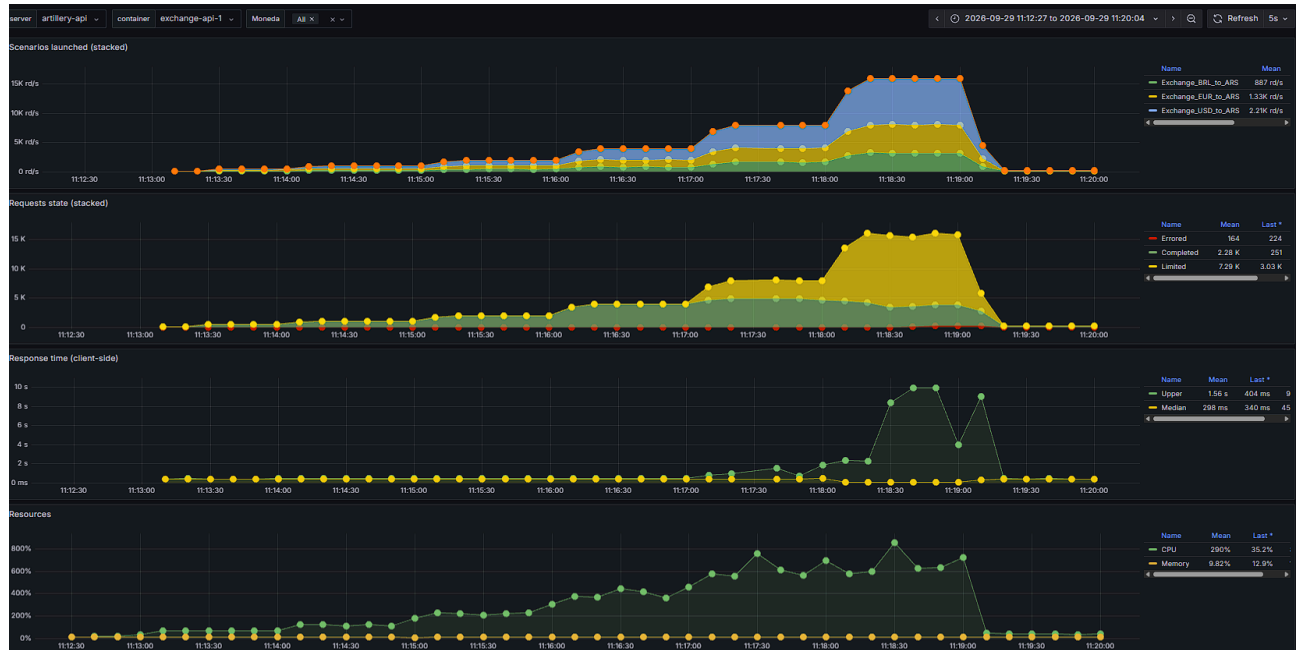
Con las protecciones (Nginx + Express) temporalmente destapadas, para medir el techo real:

Métricas	Blob único (antes)	Claves por cuenta (ahora)
Éxitos (200)	50.221	115.514 (+130%)
Fallidas	135.593 (71%)	75.136 (39%)
ETIMEDOUT	49.774	1.706 (-97%)

502 / EADDRINUSE

4.836 / 12.077

0 / 0



Qué se puede afirmar

El cambio de modelo de datos, duplicó la capacidad real medida del sistema. El nuevo techo quedó entre 500 y 600 rps.

Trade-off

Gana	Pierde / empeora
Escalabilidad	Simplicidad
Performance	Modificabilidad
Disponibilidad	

Conclusión del caso

Predicción confirmada con evidencia fuerte, el verdadero cuello de botella no estaba en el rate limiting ni en la infraestructura de red, sino en la forma en que se almacenaban los datos en Redis.

Components & Connectors

El cambio es solo a nivel de conector interno: el conector api↔Redis para transferencias deja de ser "1 comando EVAL sobre 1 key global" y pasa a ser "1 comando EVAL sobre 2 keys específicas por operación", más dos índices auxiliares (accounts:index,

accounts:by_currency:*usados por las lecturas. No se agregan componentes nuevos (siguen siendo api y Redis), pero cambia la forma de la interacción entre ellos.

Táctica 9: Tests

Motivación

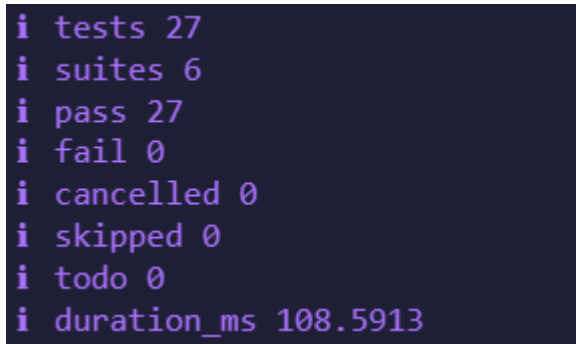
La aplicación no tenía pruebas automatizadas. Toda la validación de las tácticas anteriores se hizo a mano: correr Artillery, mirar Grafana, pegar un ``curl /accounts`` y leer el JSON. Eso tiene dos problemas concretos, no solo teóricos:

1. Es lento y no repetible. Cada verificación de corrección implica levantar toda la infra, correr 30-60 s de carga y leer números a mano. Repetirlo en cada cambio de código es inviable.
2. Hay caminos de código que ninguna carga real puede ejercitar. Las dos ramas de reconciliación de `exchange()` (`inflowOk && !outflowOk / !inflowOk && outflowOk`, cobrado-sin-pagar y pagado-sin-cobrar) estaban escritas pero nunca se disparaban, porque `transfer()` es un stub que siempre resuelve `true` — no hay forma de que Artillery, por más carga que genere, fuerce una falla que el propio código de producción no permite que ocurra. Esto no es un problema de "faltó una prueba de carga más", es estructural: sin poder controlar `transfer()` desde afuera, esas ramas son código muerto para cualquier batería de carga posible.

Táctica aplicada

Aumentar la testeabilidad del sistema:

- Suite de tests automatizados, con `node:test` (nativo de Node 24, sin dependencias nuevas) para unitarios, y `supertest` para HTTP:
- 27 tests unitarios (`app/test/unit/`) — `AccountsService`, `RatesService`, `LogService`, `ExchangeService` — cada uno con su repositorio fake en memoria, sin tocar Redis ni la red.



```
i tests 27
i suites 6
i pass 27
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 108.5913
```

- 30 tests de integración (`app/test/integration/`) — contra un Redis real (contenedor descartable) y el Express app real levantado en memoria vía `supertest`, cubriendo las 5 rutas (`/exchange`, `/accounts`, `/rates`, `/log`, `/health`) y el repositorio de `accounts` (incluyendo los scripts Lua de transferencia atómica).

```
i tests 30
i suites 8
i pass 30
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 4523.6435
```

Trade-off

- A favor: detección de regresiones rápida y repetible (segundos, no minutos), cobertura de casos de borde y combinaciones de estado que ninguna prueba de carga puede alcanzar, y verificación determinística de invariantes de concurrencia que antes solo se podían "demostrar" probabilísticamente con Artillery.
- En contra: la superficie del código productivo creció un poco (un parámetro extra en ExchangeService, un archivo nuevo createApp.js) — más piezas movibles a cambio de poder controlarlas desde un test. También hay costo de mantenimiento: los tests hay que mantenerlos sincronizados con cada cambio futuro, y una suite de integración que depende de Redis real agrega un requisito de entorno que los tests puramente unitarios no tienen.
- No hay trade-off de corrección ni de performance: ningún test cambia comportamiento en producción, y la suite no se ejecuta como parte del request path.

Conclusión del caso

Mejora clara en testabilidad, con costo bajo y acotado. Se pasó de cero tests a 57 (27 unitarios + 30 de integración), corriendo en ~4,5 s totales, sin frameworks pesados y sin cambiar el comportamiento por default del sistema.

Táctica 10: Documentación

Motivación

Contar con información acerca de la aplicación para evitar tener que adivinar o leer `app/api/*.js` para saber como funciona o integrarse correctamente.

Táctica aplicada

Documentar la interfaz (doc/API.md): para cada uno de los 7 endpoints (GET /health, GET /accounts, PUT /accounts/:id/balance, GET /rates, PUT /rates, POST /exchange, GET /log) se documentó:

- Método, path, y todos los parámetros (path, query o body) con tipo y si son requeridos.

- Todas las respuestas posibles con su status code exacto.
- Las dos capas de rate limiting (nginx limit_req_zone a 500 req/s con ráfaga de 100, y Express express-rate-limit con 500 req/s en /exchange y 300 req/s en /log), que antes no estaban documentadas en ningún lado.
- El cache de 1 segundo de GET /log en nginx y el header X-Cache-Status que lo expone.
- Todos los ejemplos de request/response del documento son capturas reales: se levantó el servicio (node app.js, contra Redis real) y se le pegó con curl a cada endpoint en cada uno de sus casos (éxito, error de validación, rechazo de negocio, rate limit) para transcribir la respuesta tal cual la devolvió el servicio corriendo.

Qué se puede afirmar

- El contrato de los 7 endpoints está ahora documentado de forma completa y verificado contra el comportamiento real del servicio corriendo.

Trade-off

A favor: reduce fricción de integración sin tocar código de producción — es puro costo de escritura, cero riesgo de regresión funcional. Hace explícitos supuestos que antes vivían implícitos en el código (status codes no convencionales, rate limits en dos capas, casos de obs: null), que son exactamente el tipo de cosa que rompe integraciones cuando se descubren en producción en vez de en un documento.

En contra: costo de mantenimiento manual — no hay ningún mecanismo que fuerce a mantener doc/API.md al día con el código; con el tiempo puede desincronizarse igual que cualquier documentación que no se valida automáticamente.

Conclusión del caso

Mejora de interoperabilidad sin costo de corrección ni de performance. El mayor valor no es solo la cobertura (7/7 endpoints) sino la exactitud: cada ejemplo es una respuesta real capturada con curl, y el proceso de escribirlos encontró matices del contrato (mensajes de error distintos según el tipo de validación, status codes no convencionales) que ni el código ni una prueba de carga hacen evidentes por sí solos. El límite claro es que esto es documentación estática — sin un mecanismo de validación automática, puede desactualizarse igual que cualquier README.

Métricas de transacciones

Siendo un requisito obligatorio se agregaron las métricas durante las transacciones.

Esto significa que cada vez que un exchange se completa con éxito, el servicio manda dos números a StatsD, separados por moneda:

1. Volumen: cuánto se operó de esa moneda, sin importar si fue compra o venta (siempre suma).

Por ejemplo, si alguien cambia 100 USD, se suman 100 al volumen de USD y se suman los ARS que salieron al volumen de ARS.

2. Neto: la plata que entra suma, la que sale resta.

Siguiendo el ejemplo: USD suma +100 (entró a la operación como origen), y ARS resta (salió de la cuenta interna hacia el cliente).

Estas dos métricas se calculan en exchange.js, en una función (recordVolume) que se llama automáticamente después de cada operación exitosa.

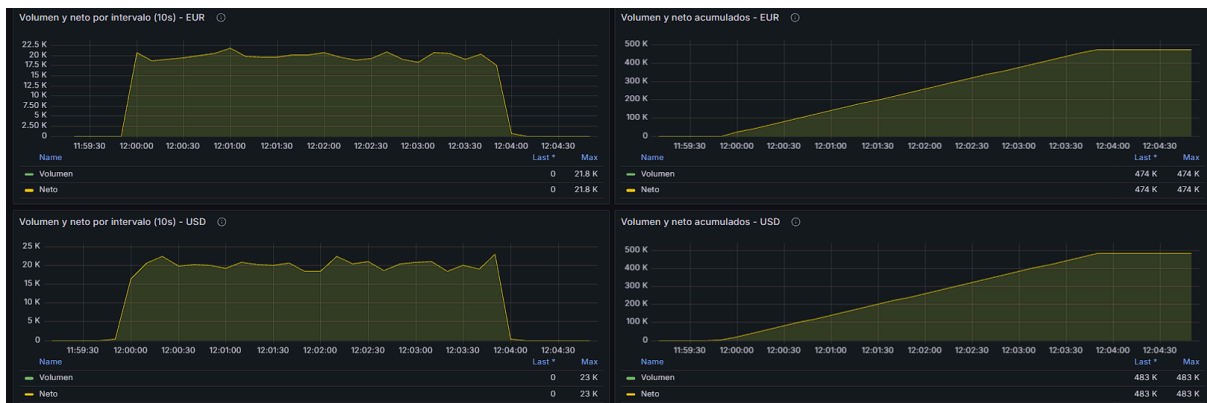
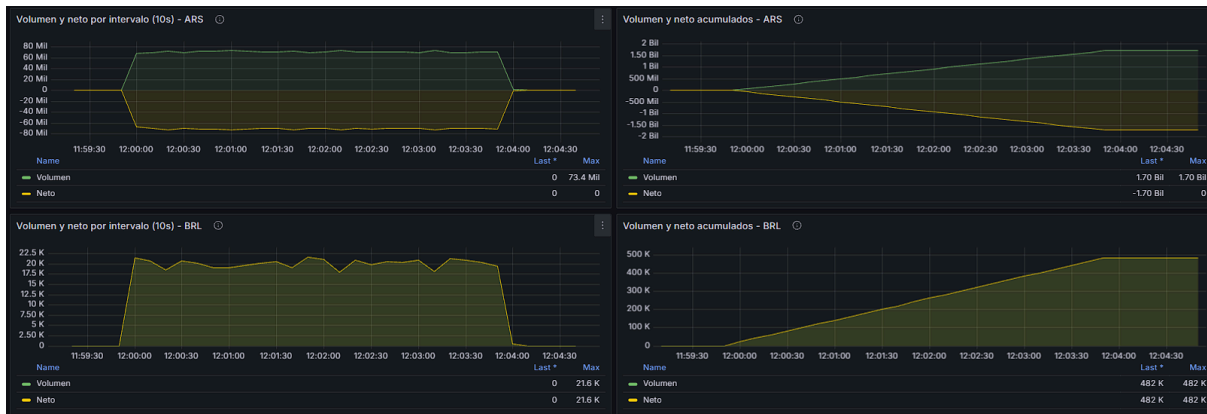
Los gráficos que se pueden observar en grafana van a mostrar

- El volumen y neto por intervalo (10s): cuánto se operó en cada ventana de 10 segundos. Así podemos ver como va variando momento a momento.
- El volumen y neto acumulados: Esta quizás es la más interesante ya que le permite a la empresa ver día a día cuanto capital se movió y los rangos horarios.

Cómo se generaron los datos que se ven en el gráfico

Para generar los gráficos primero se corrió un test de carga que hizo exchanges entre USD, EUR y BRL (todos contra ARS).

Resultado: Viendo las gráficas notamos que se ven las tres líneas de volumen subiendo parejo, y ARS con la línea de neto negativa (porque en cada operación sale ARS de la cuenta interna) mientras USD/EUR/BRL quedan con neto positivo (porque son las monedas que "entran"). Cabe aclarar que estos datos se generaron con el sistema en la táctica 8, ya que esta fue la última que generó un cambio en el sistema significativo.



Declaración de Uso de Herramientas de Inteligencia Artificial

En cumplimiento con los lineamientos establecidos para la entrega de este Trabajo Práctico, el equipo declara haber utilizado herramientas de Inteligencia Artificial como apoyo durante el desarrollo y análisis del proyecto. Específicamente, se emplearon Claude y el entorno de desarrollo Cursor con los siguientes propósitos:

- Estructuración y redacción del informe: Se utilizó Claude para asistir en el formateo del documento, organizar las secciones de manera coherente y mejorar la claridad en la redacción y estructura general del informe.
- Análisis de métricas e infraestructura: Nos apoyamos en Claude como herramienta de consulta para comprender en profundidad la interpretación de las métricas obtenidas durante las pruebas de carga y estrés, así como para resolver dudas sobre la configuración y el funcionamiento de Grafana y el resto del stack de observabilidad.
- Desarrollo y refactorización de código: Se utilizó Cursor para agilizar el desarrollo, refinar detalles de implementación y resolver cuestiones específicas de sintaxis.

asociadas a las distintas tecnologías aplicadas en la arquitectura (Node.js, scripts en Lua para Redis, etc.), manteniendo siempre el control y la revisión humana sobre la lógica de negocio final.

Conclusión

El análisis y la evolución de la arquitectura de arVault demostraron de manera práctica que el desarrollo de software orientado a atributos de calidad requiere mucho más que simplemente agregar infraestructura. El sistema original, construido bajo la premisa de salir rápido al mercado (*fake it until you make it*), presentaba deficiencias estructurales graves que amenazaban la viabilidad del negocio: condiciones de carrera que corrompían los saldos, cuellos de botella por operaciones secuenciales innecesarias y un estado volátil acoplado a un único proceso.

A través de las distintas tácticas aplicadas, logramos transformar este MVP frágil en un servicio robusto, equilibrando los atributos de calidad clave: Consistencia, Performance y Disponibilidad.

- **Consistencia ante todo:** En un entorno financiero, la integridad de los datos es innegociable. La implementación de exclusión mutua (primero en memoria y luego distribuida con Redis) resolvió las condiciones de carrera. El trade-off asumido fue una limitación en el throughput máximo sobre un mismo par de cuentas, un costo aceptable e indispensable para garantizar que nunca se generen saldos inconsistentes.
- **Performance y el falso mito de la infraestructura:** Logramos reducir la latencia a la mitad paralelizando operaciones independientes, y evitamos el colapso del sistema implementando caché y paginación en el endpoint del log. Sin embargo, el mayor aprendizaje surgió al escalar horizontalmente con Redis: añadir réplicas y externalizar el estado empeoró el rendimiento inicial debido a la contención sobre un único registro (blob). La verdadera mejora de performance ocurrió al rediseñar y particionar el modelo de datos en Redis, duplicando la capacidad real del sistema.
- **Disponibilidad y Resiliencia:** La incorporación de Rate Limiting a nivel del proxy (Nginx) demostró cómo proteger al sistema de picos de estrés, transformando colapsos caóticos (caídas y timeouts prolongados) en rechazos controlados (HTTP 429) que preservan el funcionamiento para el resto de los usuarios.
- **Observabilidad y Mantenibilidad:** La modularización del código, la creación de suites de tests automatizados, la redacción del contrato de la API y la incorporación de métricas de negocio (volumen y netos) dejaron al sistema preparado no solo para operar hoy, sino para ser mantenido y escalado a futuro.

En conclusión, las decisiones arquitectónicas tomadas y validadas empíricamente mediante pruebas de carga permitieron sanear la deuda técnica de arVault. El sistema actual ya no es

un monolito con estado en RAM: el estado de cuentas está externalizado en Redis, lo que deja preparado el camino para escalar horizontalmente.